Contents

Abstract

This report summarizes the results of several phases of the project. It is divided into two large parts. In the first part aspects of the Mixed Newton Method (MNM) in general are explored. In particular, the focus is on the minimization of analytic functions of real arguments, which are considered as functions of complex arguments by analytic continuation. The second part is devoted to the minimization of a concrete model, whose difficulty consists in the presence of continuous as well as discrete parameters. While the continuous optimization is accomplished by the Mixed Newton Method, the treatment of the discrete part is not obvious. Different approaches have been tried and are described in the second part of the report.

Summary of Section 1

One of the tasks of the project was to explore the capabilities of the MNM to minimize analytic functions of real parameters. To this end, the analytic function is continued to complex space and treated as a function of complex parameters. A restriction is that the MNM minimizes squares of analytic functions, so in order for the minimum of the square to be a minimum of the function the function has to be nonnegative. Another problem is that the modulus of a single holomorphic function cannot have local minima other than zero, so the main advantage of the MNM, the convergence exclusively to minima, seems to be lost and even a hindrance. To remedy this situation a modification of the MNM was proposed, the *regularized* MNM (RMNM). It acts in a threefold way

- the mixed Hessian is regularized to be positive definite
- local minima in real space, which are saddle points in complex space, are turned to minima again
- incursions into the complex plane are penalized

In Section 1.2 we explore the effects of general regularizations of the MNM. The main result is that arbitrary positive definite regularizations do not destroy the convergence properties, as long as they are time independent. In Section 1.3 a special cos-hyperbolic regularizer is developed for minimization of functions of real arguments. Initial experiments, shown in Section 1.5, indicated a strong preference of the method to converge to the *global* minimum. Clearly the real picture cannot be that optimistic, since the method is applicable to quite general non-convex minimization problems. In Section 1.4 the observed phenomenon is studied more thoroughly and a formula for the iteration update in this special situation is deduced. It turns out that the RMNM for the minimization of real functions with cos-hyperbolic regularizer is reduced to a gradient descent method with step size dependent on both the level of the function at the local minimum and the regularization parameter. The main outcome is as follows

- lower values of the minimum favor convergence to the minimum, but introduce chaos in the vicinity of higher-valued minima
- higher values of the regularizer suppress chaos, but equalize the minima in terms of convergence

This picture is confirmed by further experiments which are presented in Section 1.6. An algorithm that remedies all problems therefore likely does not exist.

Finally, in Section 2 we consider a different topic, namely the possibility to speed up the MNM by a Hessian free computation using only matrix-vector products instead of matrices. It is not related to RMNM

Summery of Section 3

One of the tasks of the project was to investigate a concrete precoder model with complex-valued parameters. This model involved continuous as well as discrete parameters, where the discrete parameters appeared as delays. For a given set of delays, the problem boils down to a continuous optimization problem which was solved by variants of the MNM. The choice of the delays appeared as a discrete upper level optimization problem. Different approaches have been tried for both levels of the problem and are described in the subsections of Section 3.

In Section 3.1 the model is presented and its simplest properties are studied. In Section 3.2 the lower level continuous problem is studied and several methods for its solution compared. These methods are basically different variants of the MNM. It turned out that sophisticated variants like the Cubic Newton analog do not work at all. The block simplified Cubic Newton, which is essentially a block coordinate descent by the Cubic Newton method, worked no very well. The best methods were the classical ones, damped MNM and Levenberg-Marquardt regularized MNM. The discrete optimization part is treated in Section 3.3, where several methods for delay choice are compared, namely

- cross-entropy method
- model reduction approach
- continuous interpolation (with sinc and softmax)
- sequential optimization
- local search

No one of the tested methods showed really superb performance. However, it can be said that cross-entropy should not be considered due to slow convergence, while continuous interpolation does not work well due to a complicated cost function landscape, in which the values at fractional delays do not reflect the quality of the nearest integer delays. These methods fall behind the baseline, while the model reduction approach is capable of reaching it. The sequential optimization approach can improve upon the interpolation approach, but reaches the baseline only in a part of the cases. The local search procedure, on the contrary, reached and slightly outperformed the baseline except the largest models.

As a conclusion, while no real breakthrough has been observed, the best combination seems to be damped or LM regularized MNM combined with model reduction and local search for delay optimization.

1 Regularized Mixed Newton Method: general theory

In this section we study properties and describe experimental results on the regularized mixed Newton method (RMNM). It is a development of the mixed Newton method (MNM) which was the subject of preceding projects.

1.1 Mixed Newton Method – theory and extensions

In this section we briefly recall the formulation and properties of the MNM and motivate its extension by regularization.

The MNM is used for minimization of functions of the form

$$f(z) = \sum_k |g_k(z)|^2,$$

where the functions g_k are holomorphic in the complex variable $z \in \mathbb{C}^n$. It is an iterative method starting at a point $z^0 \in \mathbb{C}^n$ and generating iterates according to the formula

$$z^{k+1} = z^k - \left(\frac{\partial^2 f(z^k)}{\partial \bar{z} \partial z}\right)^{-1} \frac{\partial f(z^k)}{\partial \bar{z}}, \tag{1}$$

where the derivatives are defined as Wirtinger derivatives and evaluated at the current point z^k.

The main advantages of the method are summarized in the following theorem.

Theorem 1.1. *The mixed derivative used in the MNM can be computed by the formula*

$$\frac{\partial^2 f}{\partial \bar{z} \partial z} = \sum_k \overline{\frac{dg_k}{dz}} \frac{dg_k}{dz}^T$$

and is hence positive semi-definite.

Let $\hat{z}$ be a critical point of the function f, i.e., $\frac{\partial f(\hat{z})}{\partial \bar{z}} = 0$. In the non-degenerate case (when the full Hessian $\frac{\partial^2 f(\hat{z})}{\partial (z,\bar{z})^2}$ is invertible) the iterates behave for z near $\hat{z}$ asymptotically as if driven by a linear dynamical system, $z^{k+1} = \hat{z} + L(z^k - \hat{z}) + O(\|z^k - \hat{z}\|^2)$, where L is an $\mathbb{R}$-linear operator. If $\hat{z}$ is a local minimum of f, then L is contracting. If $\hat{z}$ is a saddle point of f, then L has repulsive directions.

This means that the minima of f are surrounded by attraction basins, while convergence to saddle points, if it is possible at all, is unstable with respect to small perturbations of the iterates. Another advantage of the MNM, if compared to the full Newton method, is that only $\frac{1}{4}$ of the Hessian matrix has to be computed.

The MNM has been used for parameter estimation in predecoder models with complex signals. It turned out that due to symmetries in the model, i.e., (continuous) transformations of $\mathbb{C}^n$ which leave f invariant, the mixed Hessian can be degenerate. This can be countered by adding a regularizing term, and it was proven that in the cases encountered in the preceding project, a careful choice of this regularizer in theory does not alter the sequence of iterates. Another experimental observation, when minimizing arbitrary sums of squares of holomorphic functions, was that far away from critical points the iterations can lead to large jumps and erratic behaviour.

In this report we consider the effects regularization has on the convergence properties of the MNM more thoroughly. We show that the statements of Theorem 1.1 remain unaltered even with an arbitrary positive definite regularizing matrix. A second topic is the use of the MNM for the minimization of real analytic functions on $\mathbb{R}^n$, after extending them to $\mathbb{C}^n$. Here we propose a regularization which fulfills at the same time the role of a penalty pushing the minimizers towards the real subspace $\mathbb{R}^n \subset \mathbb{C}^n$.

1.2 Mixed Newton method with regularization

Instead of the MNM iterate (1) we shall consider the iterate

$$z^{k+1} = z^k - \left(\frac{\partial^2 f(z^k)}{\partial \bar{z} \partial z} + P\right)^{-1} \frac{\partial f(z^k)}{\partial \bar{z}}, \tag{2}$$

where $P \succ 0$ is a fixed positive definite complex hermitian regularizing matrix. It assures that the regularized mixed Hessian is always positive definite and can also be used for damping the step size when chosen large enough. We shall now compute the behaviour of (2) in the neighbourhood of a critical point $\hat{z}$.

Recall that we minimize a function $f(z) = \sum_i |g_i(z)|^2$ with all g_i holomorphic functions. Set $B = \frac{\partial^2 f}{\partial \bar{z} \partial z} = \sum_i \overline{g_i'}(g_i')^T$, $A = \sum_i g_i \overline{g_i''}$, where the derivatives are evaluated at the critical point $\hat{z}$ satisfying $\frac{\partial f}{\partial \bar{z}} = \sum_i g_i \overline{g_i'} = 0$. Note that $B = B^* \succeq 0$, $A = A^T$, i.e., B is complex-hermitian positive semi-definite, and A is symmetric, but in general not hermitian.

Let us now recall the proof of Theorem 1.1 in the case of the MNM iterate. Set $\delta = z - \hat{z}$, then we have for z close to $\hat{z}$ that

$$g_i(z) = g_i(\hat{z}) + (g_i'(\hat{z}))^T \delta + O(\|\delta\|^2),$$
$$g_i'(z) = g_i'(\hat{z}) + g_i''(\hat{z})\delta + O(\|\delta\|^2),$$

and therefore

$$\begin{aligned}
\frac{\partial f(z)}{\partial \bar{z}} &= \sum_i g_i(z)\overline{g_i'(z)} = \sum_i (g_i(\hat{z}) + (g_i'(\hat{z}))^T \delta + O(\|\delta\|^2))(\overline{g_i'(\hat{z}) + g_i''(\hat{z})\delta + O(\|\delta\|^2)}) \\
&= A\bar{\delta} + B\delta + O(\|\delta\|^2), \\
\frac{\partial^2 f(z)}{\partial \bar{z} \partial z} &= \sum_i \overline{g_i'(z)}(g_i'(z))^T = B + O(\|\delta\|).
\end{aligned}$$

Then the regularized MNM iterate (2) acts on the differences $\delta_k = z^k - \hat{z}$ as

$$\begin{aligned}
\delta_{k+1} &= \delta_k - \left(\frac{\partial^2 f(z^k)}{\partial \bar{z} \partial z} + P\right)^{-1} \frac{\partial f(z^k)}{\partial \bar{z}} = \delta_k - (B + P + O(\|\delta\|))^{-1} (A\bar{\delta}_k + B\delta_k + O(\|\delta_k\|^2)) \\
&= \delta_k - (B+P)^{-1}(A\bar{\delta}_k + B\delta_k) + O(\|\delta_k\|^2) = \left(I - (B+P)^{-1}B\right)\delta_k - (B+P)^{-1}A\bar{\delta}_k + O(\|\delta_k\|^2).
\end{aligned}$$

Setting $P = 0$ we obtain the original MNM. Note that in this case the first summand on the right-hand side disappears altogether if B is invertible.

Let now $H_{\mathbb{R}} = \frac{\partial^2 f(\hat{z})}{\partial (Re\, z, Im\, z)^2}$ be the Hessian at the critical point in real coordinates $(Re\, z, Im\, z)$. Then the signature of $H_{\mathbb{R}}$ is responsible for the type of the critical point. If $H_{\mathbb{R}} \succ 0$, then $\hat{z}$ is a minimum, and if the signature is mixed (with positive and negative eigenvalues), then the point $\hat{z}$ is a saddle point of the cost function f.

The matrix $H_{\mathbb{R}}$ can be transformed to the full Hessian matrix of Wirtinger derivatives by a conjugation with a non-degenerate coordinate change matrix. The Wirtinger derivatives, in turn, are given by the blocks A, B defined above. We have

$$M := \begin{pmatrix} \bar{B} & \bar{A} \\ A & B \end{pmatrix} = \begin{pmatrix} \frac{\partial^2 f}{\partial z \partial \bar{z}} & \frac{\partial^2 f}{\partial \bar{z}^2} \\ \frac{\partial^2 f}{\partial \bar{z}^2} & \frac{\partial^2 f}{\partial \bar{z} \partial z} \end{pmatrix} = \frac{1}{4} \begin{pmatrix} I & -iI \\ I & iI \end{pmatrix} \begin{pmatrix} \frac{\partial^2 f}{\partial Re\, z^2} & \frac{\partial^2 f}{\partial Re\, z \partial Im\, z} \\ \frac{\partial^2 f}{\partial Im\, z \partial Re\, z} & \frac{\partial^2 f}{\partial Im\, z^2} \end{pmatrix} \begin{pmatrix} I & -iI \\ I & iI \end{pmatrix}^*,$$

It follows that the signature of the matrix M on the left decides which type of critical point is represented by $\hat{z}$. Clearly maxima are not possible, because by virtue of $B \succeq 0$ the matrix M cannot be negative definite.

We wish to put the signature of M into relation with the stability of the dynamic system on the difference δ_k. We have the extended dynamics

$$\begin{pmatrix} \delta_{k+1} \\ \bar{\delta}_{k+1} \end{pmatrix} = \begin{pmatrix} I - (B+P)^{-1}B & -(B+P)^{-1}A \\ -(\bar{B}+\bar{P})^{-1}\bar{A} & I - (\bar{B}+\bar{P})^{-1}\bar{B} \end{pmatrix} \begin{pmatrix} \delta_k \\ \bar{\delta}_k \end{pmatrix} + O(\|\delta_k\|^2) =: Y \begin{pmatrix} \delta_k \\ \bar{\delta}_k \end{pmatrix} + O(\|\delta_k\|^2).$$

Recall that the dynamics is stable (the iterates δ_k always converge to zero) if the spectrum of Y is contained in the open unit disc.

The signature of a Hermitian matrix M does not change under transformations of the type $M \mapsto CMC^*$, while the stability of the dynamical system $x_{k+1} = Yx_k$ defined by a matrix Y does not

change under transformations of the type $Y \mapsto FYF^{-1}$, where C, F are arbitrary non-degenerate. Let $W = (B+P)^{1/2} = W^*$. Then $B = W^2 - P$. We have

$$\begin{pmatrix} \bar{W}^{-1} & 0 \\ 0 & W^{-1} \end{pmatrix} M \begin{pmatrix} \bar{W}^{-1} & 0 \\ 0 & W^{-1} \end{pmatrix} = \begin{pmatrix} I - \bar{W}^{-1}\bar{P}\bar{W}^{-1} & \bar{W}^{-1}\bar{A}W^{-1} \\ W^{-1}A\bar{W}^{-1} & I - W^{-1}PW^{-1} \end{pmatrix}$$
$$= \begin{pmatrix} I - \bar{Q} & \bar{S} \\ S & I - Q \end{pmatrix} =: M',$$

$$\begin{pmatrix} 0 & \bar{W} \\ W & 0 \end{pmatrix} Y \begin{pmatrix} 0 & W^{-1} \\ \bar{W}^{-1} & 0 \end{pmatrix} = \begin{pmatrix} \bar{W}^{-1}\bar{P}\bar{W}^{-1} & -\bar{W}^{-1}\bar{A}W^{-1} \\ -W^{-1}A\bar{W}^{-1} & W^{-1}PW^{-1} \end{pmatrix}$$
$$= \begin{pmatrix} \bar{Q} & -\bar{S} \\ -S & Q \end{pmatrix} =: Y',$$

where $S = W^{-1}A\bar{W}^{-1} = S^T$, $Q = W^{-1}PW^{-1} = Q^* \succ 0$. Note that the images of both M, Y under these transforms are now Hermitian.

The dynamical system defined by Y' is strictly stable if and only if its spectrum is contained in the interval $(-1, 1)$. This is equivalent to the condition that the spectrum of $M' = I - Y'$ is contained in the interval $(0, 2)$, in which case $M' \succ 0$. Hence stability of the dynamical system defined by the RMNM in the vicinity of a critical point implies that the point is a minimum. This still leaves open the possibility that some minima are repelling, namely, if the spectrum of M' is positive and its maximum eigenvalue exceeds 2.

Let us prove that this situation is not possible. Assume that $M' \succ 0$. Then the diagonal blocks of M' are also positive definite. In particular, we get $0 \prec Q \prec I$, where the first inequality holds by definition of Q. Hence Q and $I - Q$ are contracting, and $(I-Q)^{-1/2}, (I-\bar{Q})^{-1/2}$ are expanding, i.e., application of these maps strictly increases the 2-norm of any non-zero vector. From $M' \succ 0$ it follows that $\sigma_{\max}((I-Q)^{-1/2}S(I-\bar{Q})^{-1/2}) < 1$, i.e., this matrix product is contracting. But then S must be contracting, and all 4 blocks of the matrix M' are contracting. It follows that $\lambda_{\max}(M') = \sigma_{\max}(M') < 2$. The case of repelling minima is hence excluded.

We obtain the following result.

Theorem 1.2. *The dynamical system defined by the regularized MNM in the neighbourhood of a critical point is strictly stable if and only if this critical point is a minimum with positive definite Hessian. This dynamical system has repulsive directions if and only if the critical point is a saddle point. The case of a maximum is excluded.*

1.3 Minimization of real analytic functions

In this section we study how the MNM can be used to minimize a real analytic function F on a simply connected domain $D_{\mathbb{R}} \subset \mathbb{R}^n$. The idea is to extend the function to a domain $D_{\mathbb{C}} \subset \mathbb{C}^n$ in complex space, to yield a function of the form $|g|^2$, where $g : D_{\mathbb{C}} \to \mathbb{C}$ is a holomorphic function.

Clearly this is only possible if $F > 0$ on the domain $D_{\mathbb{R}}$. Since we may add constants to the objective, we may assume this condition without loss of generality. Then F^2 is also analytic on $D_{\mathbb{R}}$, moreover, minimizing F^2 is equivalent to minimizing F. If we define g to be the holomorphic continuation of F to a simply connected domain $D_{\mathbb{C}}$, then F^2 will be the restriction of a function $f : D_{\mathbb{C}} \to \mathbb{R}$ of the desired form $f = |g|^2$ with g holomorphic.

This straightforward approach meets the following obstacles. The sum $\frac{\partial^2 f}{\partial \bar{z} \partial z} = \sum_k \overline{\frac{dg_k}{dz}} \frac{dg_k}{dz}^T$ defining the mixed Hessian consists of only one summand, hence the mixed Hessian is rank 1. Another problem is that the minima of f can be located in the complex plane, and a local minimum of f on $D_{\mathbb{R}}$ may be a saddle point of f on $D_{\mathbb{C}}$. The iterates will then be pushed away from $D_{\mathbb{R}}$.

Here we propose an approach which remedies both problems. It consists in adding to f a sum of squares of holomorphic functions which penalizes deviations of z into the complex plane, but does not alter the values of f on $D_{\mathbb{R}}$. At the same time the additional summands in the mixed Hessian regularize this matrix and render it positive definite.

Suppose that $g_0 : D_{\mathbb{C}} \to \mathbb{C}$ a holomorphic function and that the goal is to minimize the modulus $|g_0|$ on the intersection $D_{\mathbb{R}} = \mathbb{R}^n \cap D_{\mathbb{C}}$. We shall minimize the function

$$\sum_{k=0}^{2n} |g_k(z)|^2$$

instead, where

$$g_k(z) = \gamma e^{iz_k}, \quad k = 1, \dots, n; \qquad g_k(z) = \gamma e^{-iz_k}, \quad k = n+1, \dots, 2n$$

with $\gamma > 0$ a weighting coefficient. Then the additional term in the objective amounts to

$$\sum_{k=1}^{2n} |g_k(z)|^2 = 2\gamma^2 \sum_{k=1}^{n} \cosh(2Im\, z_k).$$

The additional term has the following properties:

- on $\mathbb{R}^n$, hence on $D_{\mathbb{R}}$ the constant $2n\gamma^2$ is added to the objective
- for complex entries z_k the modulus of the imaginary part is exponentially penalized
- the amount of penalization can be tuned by choosing the coefficient γ

The mixed Hessian, which is initially rank 1, obtains an additive correction given by

$$\sum_{k=1}^{2n} \overline{\frac{dg_k}{dz}} \frac{dg_k}{dz}^T = 2\gamma^2 \sum_{k=1}^{n} \cosh(2Im\, z_k) e_k e_k^T \succeq 2\gamma^2 I.$$

Hence this correction acts as a positive definite regularizer.

Finally we obtain the following iteration of the MNM for minimization of a positive analytic function $F = g_0$ on $D_{\mathbb{R}}$ with additive complex-repulsive regularization:

$$z^{k+1} = z^k - \left(\overline{g_0'(z^k)} (g_0'(z^k))^T + 2\gamma^2 \sum_{j=1}^{n} \cosh(2Im\, z_j^k) e_j e_j^T \right)^{-1} \left(g_0(z^k) \overline{g_0'(z^k)} + 2i\gamma^2 \sum_{j=1}^{n} \sinh(2Im\, z_j^k) e_j \right). \tag{3}$$

Here z_j^k is the j-th element of the current point z^k, and e_j is the j-th basis vector.

Note that the ordinary Newton method without regularization, acting in real space $\mathbb{R}^n$, proceeds by the iterate

$$x^{k+1} = x^k - g_0(x^k) \left(g_0(x^k) g_0''(x^k) + g_0'(x^k)(g_0'(x^k))^T \right)^{-1} g_0'(x^k). \tag{4}$$

1.4 Behaviour of regularized MNM near real stationary points

In the previous section we described how to use the regularized mixed Newton method (RMNM) for the minimization of real analytic functions. Although it could be observed in experiments that the global minimum remained attractive and the saddle points and maxima remained repulsive, some local minima showed also repulsive behaviour. This discrepancy with the general theory can be due to different reasons:

- minima in real space do not need to be minima in complex space
- the addition of regularizing functions (amounting to the penalty term $2\gamma \sum_k \cosh(2 \cdot Im\, z_k)$ to the initial cost $g(x)^2$) can turn a saddle point into a minimum
- a saddle point of some dynamic system does not need to remain a saddle point if the dynamics is restricted to an invariant subspace

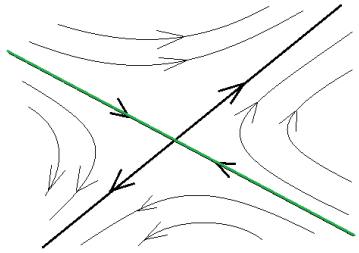


Figure 1: In the embedding space the fixed point is a saddle point, but in the green subspace it is attractive.

The last point can be visualized in Fig. 1.

We consider the cosh-regularized mixed Newton method for the minimization of real analytic functions $g(x) > 0$ for real values of the argument x. Inserting $Re\, z_k = x_k$, $Im\, z_k = 0$ into (3), we get iterates of the form

$$x_{k+1} = x_k - \frac{g(x_k)g'(x_k)}{2\gamma^2 + \|g'(x_k)\|^2}. \tag{5}$$

Here $\gamma > 0$ is the regularizing weight.

The regularizing functions are given by $\gamma e^{\pm i x_l}$, $l = 1, \ldots, n$. Their contribution to the mixed Hessian and the gradient can be evaluated explicitly, so only the gradient and the Hessian of the target function g are of interest. The overall cost function is then

$$|g(x)|^2 + 2\gamma^2 \sum_{l=1}^{n} \cosh 2Im\, x_l.$$

A real point x is stationary for f if and only if $g'(x) = 0$.

The theory of the RMNM says something about the dynamics of the method near stationary points in complex space. If the point $\hat{x}$ is a local minimum of g in the real subspace, then for large enough γ the whole cost function has a local minimum at $\hat{x}$. If $\hat{x}$ is a saddle point or maximum of g in the real subspace, then the whole cost function will have a saddle point at $\hat{x}$ for all values of γ. Hence the complex dynamics of the RMNM is always repelled from saddle points or maxima and attracted to minima for large enough values of γ.

However, since the dynamics for a real starting point are confined to the real subspace, repelling properties from the complex dynamics might be lost. Let us consider the dynamics in real space.

Let $\hat{x}$ be a stationary point of g, $g'(\hat{x}) > 0$, such that $g(\hat{x}) \neq 0$. Then $\hat{x}$ is a fixed point of the RMNM and the residual $\delta = x - \hat{x}$ evolves according to the asymptotic dynamics

$$\delta_{k+1} = \delta_k - \frac{g(\hat{x})g''(\hat{x})}{2\gamma^2}\delta_k + O(\|\delta_k\|^2).$$

The matrix of the linear part of the dynamics hence equals

$$I - \frac{g(\hat{x})g''(\hat{x})}{2\gamma^2}.$$

If $g''(\hat{x}) \succ 0$, then for large enough γ the dynamics is stable. However, for small γ stability is lost, and the steps can be large.

If $g''(\hat{x})$ is not positive semi-definite, then the dynamics is not stable for any value of γ.

Hence the stability results from complex space carry over to the dynamics in real subspace. In order to profit from the attractive property in the vicinity of minima, the parameter γ has to be chosen larger than $\frac{\sqrt{g(\hat{x})\lambda_{\max} g''(\hat{x})}}{2}$.

Let us summarize these results in the following theorem.

Theorem 1.3. *Let $\hat{x}$ be a stationary point of the function $g(x)$. Consider the regularized mixed Newton method with a real starting point. Then*

- *$\hat{x}$ is a fixed point of the RMNM*
- *if $\hat{x}$ is not a minimum, then it is not attractive for the method*
- *if $\hat{x}$ is a local minimum, then it is attractive for $\gamma > \frac{\sqrt{g(\hat{x})\lambda_{\max} g''(\hat{x})}}{2}$*

1.5 Numerical experiments

In this section, we show the results of the conducted experiments for the comparison of the RMNM (3) and the Ordinary Newton Method (4). We consider the minimization problem of positive real polynomials in $\mathbb{R}^2$, as in the following examples.

Example 1.4. In this example, we consider the following polynomial

$$f(x) = (2x_1 - 3x_2)^2 + x_1^2(1 - x_1)^2 + x_2^2(1 - x_2)^2, \tag{6}$$

where $x = (x_1, x_2) \in \mathbb{R}^2$.

The function (6) is positive non-convex. For this function, we have a global minimum at the point $(0, 0)$ with optimal value $f^* = 0$. Also, we have a local minimum at the point $\approx (1.04987, 0.709507)$ with value ≈ 0.0460496, and a saddle point at $\approx (0.577876, 0.378919)$ with value ≈ 0.11525.

Example 1.5. In this example, we consider the following polynomial in $\mathbb{R}^2$

$$f(x) = f(x_1, x_2) = (x_1 - x_2)^2 + x_1^2(1 - x_1)^2 + x_2^2(2 - x_2)^2. \tag{7}$$

The function (7) is positive non-convex. For this function, we have a global minimum at the point $(0, 0)$ with optimal value $f^* = 0$. Also, we have a local minimum at the point $\approx (1.27473, 1.81735)$ with value ≈ 0.527264593, and a saddle point at $(1, 1)$ with value $f(1, 1) = 1$.

Example 1.6. In this example, we consider the following polynomial in $\mathbb{R}^2$

$$f(x) = f(x_1, x_2) = (x_1 + 1)^4 + (x_2 + 1)^4 + 4x_1x_2. \tag{8}$$

The function (8) is positive non-convex. For this function, we have a global minimum at the point $(-0.31767219617, -0.31767219617)$ with optimal value $f^* = 0.83717564078542$. Also, we have a local minimum at the point $\approx (0.1537213755, -1.53568738679)$ with value ≈ 0.90983005625052, and another local minimum at the point $(-1.53568738679, 0.153721375541)$ with value ≈ 0.90983005625052, and a saddle point at $(-1, 0)$.

We rum RMNM and the Ordinary Newton Method, with different 625 initial points in the square $[-1, 2] \times [-1, 2]$ for Example 1.4, and with 1024 initial points in the square $[-1, 3] \times [-1, 3]$ for Example 1.5, and with different 5329 initial points in the square $[-3, 2] \times [-3, 2]$ for Example 1.6. For RMNM the initial points have the form $z_0 = (x_0 + 0i, y_0 + 0i) \in \mathbb{C}^2$, where $(x_0, y_0) \in \mathbb{R}^2$ in the correspondence previously mentioned squares, are the initial points for the Ordinary Newton Method. The results are presented in Table 1 and Table 2. These results show the number of initial points for which the methods converge to the local or global minimum, also show the number of initial points for which the methods diverge, and the number of initial points for which the methods do not converge after performing 10^6 iterations.

Remark 1.7. If we take the initial points of RMNM with the form $(x_0 + i, y_0 + i) \in \mathbb{C}^2$ where $(x_0, y_0) \in [-1, 2] \times [-1, 2]$, (i.e., the imaginary parts of all coordinates are 1, not 0) then the RMNM will work worse. See Table 3, it shows the results of the comparison between RMNM and Ordinary Newton Method for Example 1.4, where $(x_0, y_0) \in [-1, 2] \times [-1, 2] \subset \mathbb{R}^2$ are the initial points for Ordinary Newton method.

	RMNM				Ordinary Newton Method			
	# points to local min	# points to global min	# points with diverg.	# points without converg. after 1 million iterations	# points to local min	# points to global min	# points with diverg.	# points without converg. after 1 million iterations
Ex. 1.4 with 625 initial points	0	625	0	0	319	276	0	30
Ex. 1.5 with 1024 initial points	0	1024	0	0	443	463	1	117

Table 1: The results of RMNM (with $\gamma = 10^{-3}$) and Ordinary Newton Method for Examples 1.4 and 1.5.

	RMNM	Ordinary Newton Method
# points global min	5329	1381
# points local min 1	0	1806
# points local min 2	0	1806
# points with divergence	0	2
# points without convergence after 1 million iterations	0	334

Table 2: The results of RMNM (with $\gamma = 10^{-2}$) and Ordinary Newton Method for Example 1.6, with 5329 initial points.

	RMNM	Ordinary Newton Method
# points global min	25	23
# points local min	0	24
# points with divergence	0	0
# points without convergence after 1 million iterations	24	1

Table 3: The results of RMNM (with $\gamma = 10^{-3}$) and Ordinary Newton Method for Example 1.4, with 49 initial points.

1.6 Further experiments

In this section, we test RMNM for the problem of minimization of a positive real polynomial on $\mathbb{R}^n$, with different values of the parameter γ. As stated above, one should expect better attraction to minima with growing γ, but a less marked differentiation between the minima.

We again consider example 1.4 with function (6). We run RMNM with 961 different initial points in the square $[-1, 2] \times [-1, 2]$ and with different values of the parameter γ. The results of the tests are presented in Table 4, below.

γ	# points with converg. to global	# points with converg. to local	# points with converg. to saddle	# points with cycle around local
0.01	**960**	0	0	0
0.5	476	0	0	484
0.6	476	484	0	0
0.7	476	484	0	0
1	476	484	0	0
2	476	484	0	0
3	476	484	0	0
5	476	484	0	0

Table 4: The results of RMNM (3) for different values of γ and with 960 different initial points in the square $[-1, 2] \times [-1, 2]$.

These results show that RMNM doesn't converge to the saddle point (whereas the Ordinary Newton Method convergences to the saddle point for some initial point). It always convergences to the global minimum for any small enough values for γ. For some bigger γ, for example, $\gamma = 0.5$ we find that there is a part of initial points for which RMNM cycles in the neighborhood of the local minimum (by a cycle we mean that there is $k > 1$ such that $f(z^{m+2}) = f(z^m)\ \forall m \geq k$). But for a still bigger γ, we can note the disappearance of the cycles, and instead RMNM convergences to a local minimum, also it convergences to the global (the convergence to global is slow). We note also that for different big values of γ there is not a difference between results. The volumes of the attraction basins around local and global minimums don't change by changing the value of γ.

Let us summarize the findings of the experiments. Increasing γ does not change how the method discriminates between different minima. It rather switches from an indirect convergence to the local minimum, by which we mean convergence to a periodic trajectory in the vicinity of the local minimum, to direct convergence. The convergence to the global minimum remains unaltered.

Only for very small values of γ a preference of convergence to the global minimum can be seen. This is not of big practical use, however, because the converging trajectories consist of a large part of an erratic behavior until they hit the attraction basin of the global minimum.

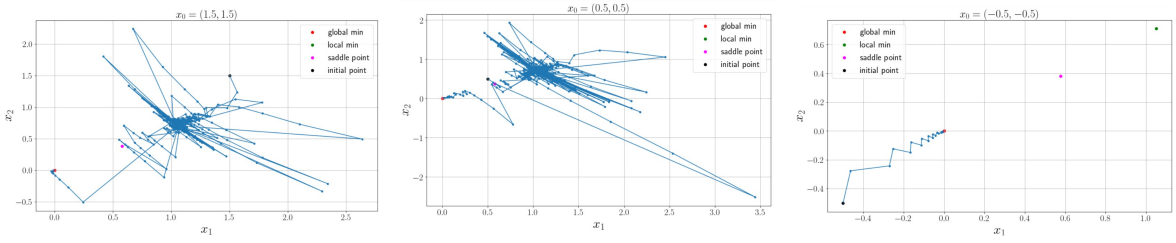


Figure 2: Trajectories of the RMNM, with $\gamma = 0.01$. Here RMNM converges to the global minimum.

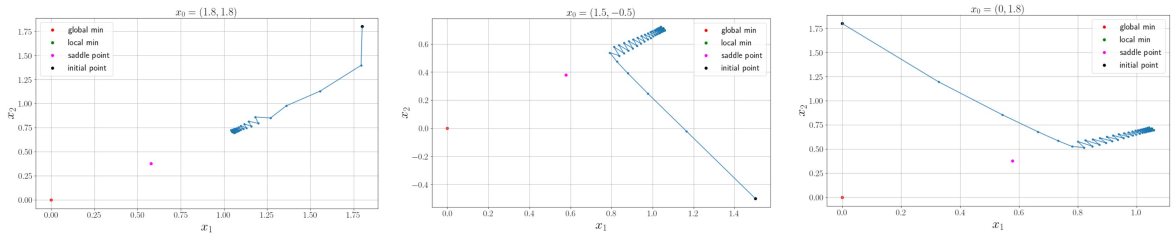


Figure 3: Trajectories of the RMNM, with $\gamma = 0.5$. Here RMNM cycles in the neighborhood of the local minimum.

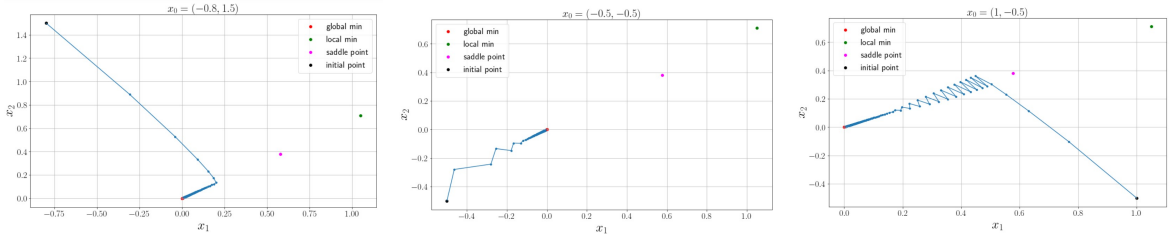


Figure 4: Trajectories of the RMNM, with $\gamma = 0.5$. Here RMNM converges to the global minimum.

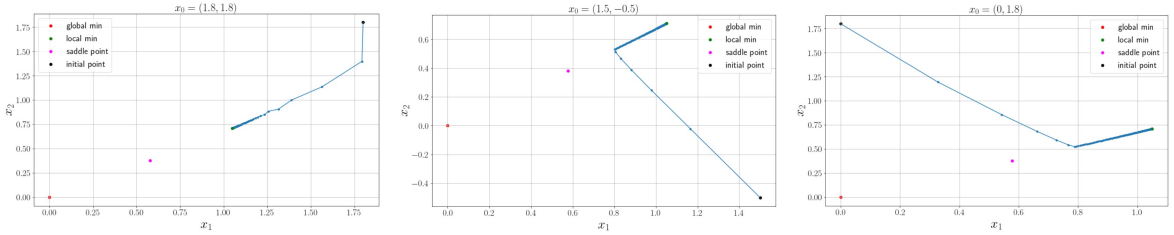


Figure 5: Trajectories of the RMNM, with $\gamma = 1$. Here RMNM converges to the local minimum.

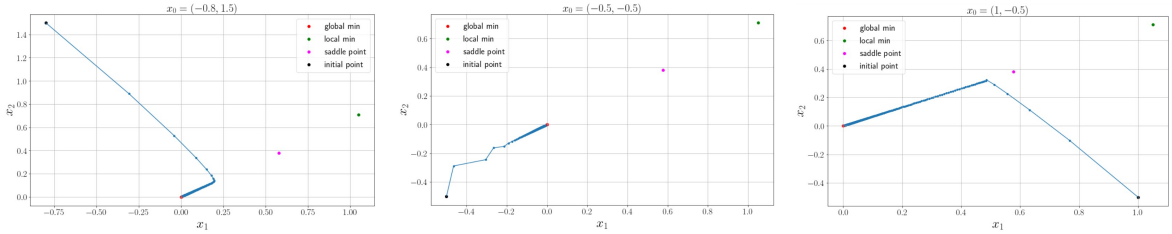


Figure 6: Trajectories of the RMNM, with $\gamma = 1$. Here RMNM converges to the global minimum.

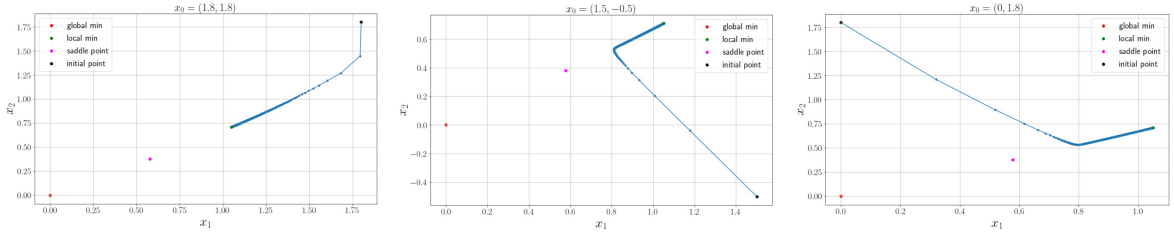


Figure 7: Trajectories of the RMNM, with $\gamma = 5$. Here RMNM converges to the local minimum.

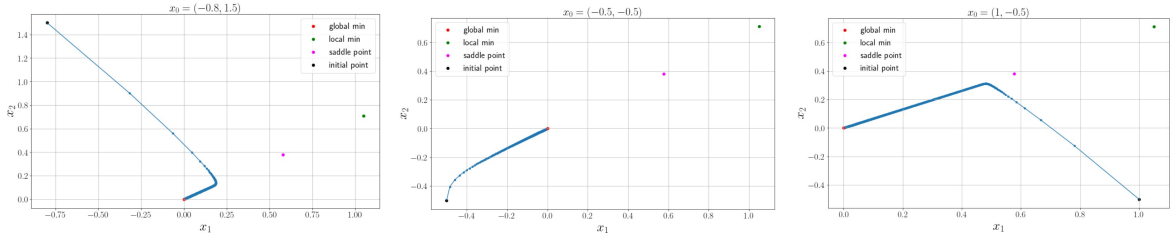


Figure 8: Trajectories of the RMNM, with $\gamma = 5$. Here RMNM converges to the global minimum.

2 Hessian free Mixed Newton method

Another topic is on a computational speed-up of the MNM. In analogy with the conjugate gradient method, we replace the solution of the system with the mixed Hessian as a matrix by a lower-level iterative process requiring only matrix-vector products involving the mixed Hessian. These matrix-vector products can, as in other Hessian-free methods, be computed by evaluating the gradient of a directional derivative. In the case of the mixed Newton, we do not need to compute second-order derivatives and can evaluate the product using first derivatives only. This is accomplished in Section 2.

2.1 Motivation

Consider the problem of minimizing the sum of squares

$$f(z) = \sum_k |g_k(z)|^2 = \|g(z)\|^2$$

of holomorphic functions $g_1, \dots, g_K$, collected in a vector-valued function g.

The mixed Newton method computes its iterations according to the formula

$$z_{k+1} = z_k - \gamma_k \left(\frac{\partial^2 f}{\partial \bar{z} \partial z} \right)^{-1} \frac{\partial f}{\partial \bar{z}}.$$

For a full step-size $\gamma_k = 1$ the iteration can be interpreted as minimizing the convex quadratic approximation

$$q(z; z_k) = \|g(z_k) + J(z_k)(z - z_k)\|^2,$$

where $J = \frac{dg}{dz}$ is the derivative of the vector g of holomorphic functions. Note that $\frac{\partial^2 f}{\partial \bar{z} \partial z} = J^* J$, $\frac{\partial f}{\partial \bar{z}} = J^* g$.

If the number of functions and the number of variables are large, then it can be unpractical to compute the mixed Hessian $M = \frac{\partial^2 f}{\partial \bar{z} \partial z}$ or the Jacobian J. In this case, approximate methods based on the conjugate gradient (Hessian-free methods) have to be used. These methods do not require inversion of the mixed Hessian or solution of a linear system involving this matrix, but only matrix-vector products.

When dealing with the ordinary Newton method and real functions, the Hessian $H = \frac{\partial^2 f}{\partial x^2}$ takes the place of the mixed Hessian. For a given vector u, the product Hu is then the derivative of the directional derivative $\nabla_u f$, or the directional derivative of the gradient of f. They can be computed in linear time by automatic differentiation techniques.

The goal is to provide a similar algorithm for a Hessian free mixed Newton method.

2.2 Complex conjugate gradient

Let us consider the problem

$$\min_{z \in \mathbb{C}^n} (z^* M z + b^* z + z^* b),$$

where $M \succ 0$ is a complex Hermitian matrix and $b \in \mathbb{C}^n$ a vector. We want to design a conjugate gradient (CG) method for this problem.

Note that we may rewrite the quadratic cost as a real quadratic cost in the real vector $(Re\, z, Im\, z)$ and apply the usual CG method to this problem. We develop a complex variant, however.

The gradient of the function $f(z) = z^*Mz + b^*z + z^*b$ is given by $\frac{\partial f}{\partial \bar{z}} = Mz + b$. We then have

$$f(z) = f(\hat{z}) + 2 \cdot Re\left(\left(\frac{\partial f(\hat{z})}{\partial \bar{z}}\right)^* (z - \hat{z})\right) + O(\|z - \hat{z}\|^2).$$

Hence the gradient points in the steepest ascent direction. The optimum is achieved when the gradient is zero, at $z^* = -M^{-1}b$.

Let z_0 be an initial vector. The CG method produces a sequence of points $z_1, \ldots, z_k, \ldots$. Denote $q_k = Mz_k + b$. In analogy the the real CG, we choose the point z_{k+1} as the minimizer of f over the subspace $z_0 + \text{span}\{q_0, \ldots, q_k\}$.

Let us introduce also a sequence $p_0, \ldots, p_k, \ldots$ such that

$$\text{span}\{q_0, \ldots, q_k\} = \text{span}\{p_0, \ldots, p_k\} \qquad \forall\ k$$

and $p_k^* M p_l = 0$ for all $k \neq l$. This condition can be fulfilled if we define

$$p_k = q_k - \sum_{l=0}^{k-1} \frac{p_l^* M q_k}{p_l^* M p_l} p_l.$$

Note that $p_0 = q_0 = Mz_0 + b$.

Set $z = z_0 + \sum_{l=0}^{k} \alpha_l p_l$, where $\alpha_l \in \mathbb{C}$. Then

$$f(z) = f(z_0) + \sum_{l=0}^{k} \left(\alpha_l (Mz_0 + b)^* p_l + \bar{\alpha}_l p_l^* (Mz_0 + b) + |\alpha_l|^2 p_l^* M p_l\right).$$

The point z_{k+1} is defined by the coefficients α_l minimizing of this function. The first-order optimality condition $\frac{\partial f}{\partial \bar{\alpha}_l} = 0$ yields

$$p_l^* p_0 + \alpha_l p_l^* M p_l = 0, \qquad \alpha_l = -\frac{p_l^* p_0}{p_l^* M p_l}.$$

This finally gives the formula

$$z_{k+1} = z_k + \alpha_k p_k = z_k - \frac{p_k^* p_0}{p_k^* M p_k} p_k.$$

The method can also be written in recursive form. We have that

$$q_{k+1} = q_k + \alpha_k M p_k, \quad \|q_k\|^2 = p_k^* p_0, \quad p_{k+1} = q_{k+1} + \frac{\|q_{k+1}\|^2}{\|q_k\|^2} p_k$$

for all k. Hence we may compute the iterates by the formulas $q_0 = p_0 = Mz_0 + b$,

$$\begin{aligned}
\xi_k &= M p_k, \\
\alpha_k &= -\frac{\|q_k\|^2}{p_k^* \xi_k}, \\
z_{k+1} &= z_k + \alpha_k p_k, \\
q_{k+1} &= q_k + \alpha_k \xi_k, \\
\beta_k &= \frac{\|q_{k+1}\|^2}{\|q_k\|^2}, \\
p_{k+1} &= q_{k+1} + \beta_k p_k.
\end{aligned}$$

Here the most expensive operation is the computation of the matrix-vector products Mz_0, Mp_k. In the next section, we consider how to compute this product in linear time using the structure of M.

2.3 Matrix-vector products

Let us consider a direction $u \in \mathbb{C}^n$. We wish to compute the product $Mu = J^*Ju$ in linear time. Recall that $J = \frac{dg}{dz}$ is the Jacobian of a vector-valued holomorphic function g.

Let us accomplish this in two steps. First, we compute

$$Ju = \nabla_u g = \lim_{t \to 0} \frac{g(\hat{z} + tu) - g(\hat{z})}{t},$$

where $\hat{z}$ is the point where the Hessian-vector product is evaluated. We thus have to compute a directional derivative of the vector-valued function g.

Set $\xi = Ju$, we then have to compute $s = J^*\xi$. We have

$$s^* = \xi^* J = \frac{d}{dz}(\xi^* g).$$

Hence to compute the complex conjugate of s, we have to compute the derivative of the scalar holomorphic function $\xi^* g$, which is a linear combination of the holomorphic functions g_k.

Hence the evaluation of a mixed Hessian vector product amounts to the computation of a directional derivative of the vector-valued holomorphic function g, and the derivative of a scalar holomorphic function. We do not need to compute second-order derivatives, even directional ones.

The computation of the matrix-vector product $s = Mu = J^*Ju$ can then be summarized as follows:

- compute the directional derivative $\xi = Ju = \nabla_u g$
- compute the derivative $v = \xi^* J = \frac{d}{dz}(\xi^* g)$
- compute $s = v^*$

In the case of an additional regularizing matrix P, i.e., when $M = J^*J + P$, one has to add the product Pu to the result. We then have to assume that Pu is easily computed, e.g., if P is proportional to the identity matrix or of low rank.

3 Optimization of two-layer model

In this section we describe the approaches to optimize a concrete two-layer precoder model and the experimental results which compare these approaches.

We compare different methods for training a 2-layer model for precoder design. The model description is provided below in Section 3.1.

The optimization of the model parameters has a discrete and a continuous component. Accordingly, we developed a 2-level algorithm for training of the model, consisting of a discrete and a continuous optimization algorithm. The discrete algorithm solves the upper level optimization problem and is designed for choosing the delays in the model. The continuous algorithm solves the lower level problem and aims at choosing the continuous parameters of the FIR filters and the mixing matrices for fixed delays.

The lower level optimization is accomplished by a variant of the Mixed Newton Method (MNM). Since the plain MNM suffers from drawbacks caused by degeneracies of the step stemming from the symmetries of the model (see Section 3.1.3), we studied modifications of the MNM. It was established theoretically that a regularization of the potentially degenerate Hessian by an arbitrary positive definite matrix preserves the favorable convergence properties of the MNM. These properties are briefly recalled in Section 1.1. Accordingly, the main tool to solve the lower level optimization problem will be the Regularized Mixed Newton Method (RMNM). Its theoretical properties are investigated in Section 1.2.

The investigations on the precoder model can be divided into two parts.

Although the lower level optimization was initially considered as the simpler part and the Mixed Newton Method (MNM) was the choice of the art for this part, our investigations showed significant differences in convergence speed for different variants of the method. We also tested some continuous non-convex optimization methods for the lower level optimization which cannot be considered as being derived from the MNM. The results of these experiments are described in Section 3.2. Section 3.3 is devoted to the more complicated problem of choosing the delays. Here also several methods were tried and compared.

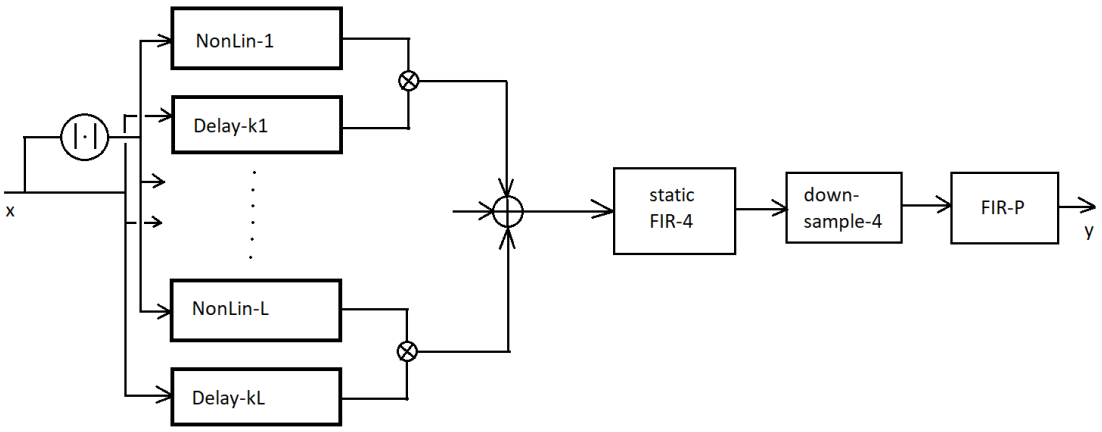


Figure 9: Upper level model architecture

3.1 Model description

In this section we describe the model used to correct the nonlinear effects of the power amplifier and whose parameters are to be optimized. The model has two sets of parameters, continuous coefficients and discrete delays. It is holomorphic in the continuous parameters and hence the Mixed Newton Method (MNM) can be used to train this subset of parameters for a fixed set of delays.

3.1.1 Input-output structure and objective

The input of the model is constituted of a sequence of scalars $x_t \in \mathbb{D} \subset \mathbb{C}$, $t = 1, \ldots, N$, where $\mathbb{D}$ is the unit complex disc. For t outside of the range $[1, N]$ the signal x_t is assumed to be zero. The output is a sequence of scalars $y_t \in \mathbb{C}$, $t \in [1, N]$, $t \equiv 0(4)$, where y_t is a function of $x_{t-l}, x_{t-l+1}, \ldots, x_{t+r}$ for certain $l, r \geq 0$ which depends on the model parameters. Note that the sample rate of the output is 4 times smaller than that of the input, and y_t is defined only for t divisible by 4. The output is compared to a measured signal $d_t \in \mathbb{C}$, $t = 1, \ldots, N$. The model parameters have to be chosen to minimize the mismatch

$$\sum_{t=l+1}^{N-r} |y_t - d_t|^2, \qquad t \equiv 0(4).$$

Hence the objective function to minimize is a sum of squared moduli of holomorphic functions of the parameters. The MNM can compute the next iterate using only the first derivatives of y_t with respect to the parameters. Note that the objective runs over an interior subinterval of the initial interval $[1, N]$ in order to avoid the boundary effects caused by the zero padding of the input signal.

3.1.2 Model description

The global architecture of the model determining the dependence of the output y_t on the input x_t is depicted in Fig. 9.

The input signal is split into $2L$ channels, gathered into L pairs. One of the signal copies in each pair is first passed through a static non-linearity, namely the complex modulus function $\mathbb{D} \to [0, 1]$, and then through a non-linear block, each having its own parameterization, and the other copy is passed through a delay of k_j samples, $j = 1, \ldots, L$. The delays k_j are part of the discrete parameter set of the model. After transformation the two output signals of each pair of blocks are multiplied together, and the resulting L products are summed. The sum is passed through a 3-th order FIR filter whose 4 coefficients are fixed and cannot be trained. The next block is a downsampling block which deletes three in four elements of the signal sequence. Finally, the downsampled sequence is passed through a FIR filter with P coefficients, yielding the output y_t.

Let us now describe the structure of the non-linear block i, whose layout is depicted in Fig. 10.

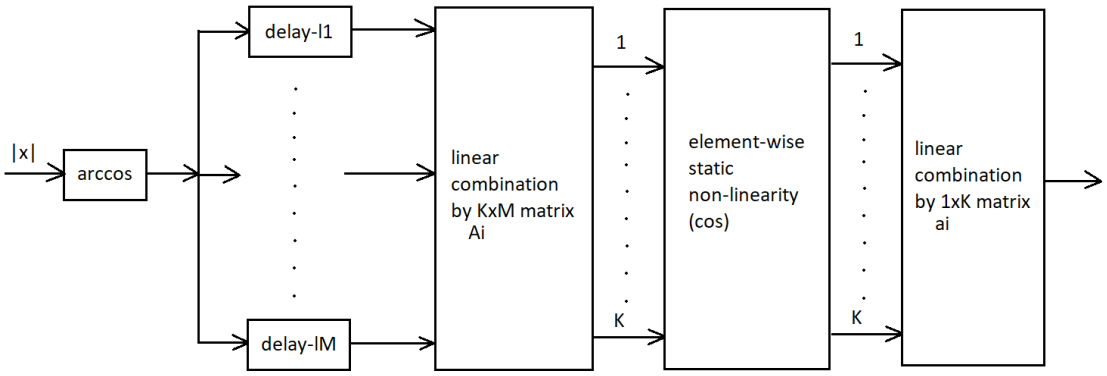


Figure 10: Architecture of the non-linear blocks

At first the modulus of x_t is passed through the arccos static non-linearity, which extends the range to $[0, \frac{\pi}{2}]$. Then the signal is split into M channels and passed through M different delays of $l_{i,j}$ samples, $j = 1, \ldots, M$. The delays $l_{i,j}$ are part of the discrete set of parameters. The delayed arccosini of the modulus are then linearly combined to K complex linear combinations, each of which is passed through a cos static non-linearity. These K signals are then linearly combined to the output of the non-linear block.

We therefore obtain a discrete parameter set consisting of the delays k_j, $j = 1, \ldots, L$ and $l_{i,j}$, $i = 1, \ldots, L$, $j = 1, \ldots, M$, in total $L(M + 1)$ discrete parameters. These are complemented by the continuous parameters in the coefficient matrices $A_1, \ldots, A_L \subset \mathbb{C}^{K \times M}$, in the vectors $a_1, \ldots, a_L \in \mathbb{C}^{1 \times K}$, and in the FIR block, in total $LK(M + 1) + P$ parameters.

Three problems have been formulated in connection with the training of this model:

- find better discrete parameters, the continuous ones being trained by the MNM
- run the MNM by forming the Hessian and gradient only from a subset of samples, chosen randomly
- find better static non-linearities than arccos and cos (mainly cos)

3.1.3 Symmetries

The model has a number of symmetries, i.e., groups acting on the parameters of the model without changing the objective value, in a manner independent of the input x.

Homogeneous scaling: If the $1 \times K$ vectors in the last layer of the NL blocks are multiplied by the same non-zero constant $\alpha \in \mathbb{C}$, and the coefficients in the FIR block are multiplied by α^{-1}, then the output stays the same. This symmetry leads to a degeneration of the MN step. This can be remedied by setting, e.g., a fixed coefficient in the FIR block to 1.

Permutations of the coefficients next to the static nonlinearity: If the K rows of the $K \times M$ coefficient matrix and the $1 \times K$ vector are permuted with the same permutation, then the output of the model does not change. This symmetry acts on the continuous parameters but does not lead to degeneration of the MN step.

Permutation of the NL-delay pairs: Since the product outputs of each NL-delay pair are summed, their permutation does not change the model output.

Permutations of the delays in the NL blocks: The outputs of the delays in each NL blocks are linearly combined with arbitrary complex coefficients. Therefore permuting the delay blocks and simultaneously permuting the columns of the $K \times M$ coefficient matrices by the same permutations does not change the output.

Cancelling of delays: If all delays k_j and $l_{i,j}$ are shifted by a number $4k$, where k is integer, and the delays in the FIR block are shifted by $-k$, then the output of the model is the same.

3.1.4 Use of symmetries

The choice of the delays and of meta-parameters such as the number of delays or the number of linear combinations to be fed to the static non-linearity is essentially a combinatorial problem which confronts the user with searching over a large number of parameter sets. The presence of discrete symmetries allows to decrease the number of parameter sets, because different sets may lead to essentially the same model.

Permutation of the NL-delay pairs: By applying an appropriate permutation we may assume that the delays k_j applied to the complex signal x_t before multiplication with the NL block output are strictly increasing.

Permutations of the delays in the NL blocks: By applying an appropriate permutation we may assume that the delays $l_{i,j}$ within each NL block i are strictly increasing. The corresponding permutation of the coefficients in the $K \times M$ matrix do not change the feasible set for the MNM optimization.

3.2 Optimization of continuous parameters

In this section, we present the methods we tested for the lower level continuous optimization and the results. The tested methods can be divided into two groups: Mixed Newton Methods and Block Methods. The last group is based on the idea of separate calculation of the Hessian or Jacobian Matrix for different blocks. As a result, we obtain a block-diagonal pre-conditioner that replaces the full matrices in the Mixed Newton Methods.

Mixed Newton Methods. First we consider the *Cubic Newton Method*. This is a now classical method which works by minimizing a third-order pseudo-polynomial at each iteration. This polynomial is majorating the objective function, which is considered as being dependent on the real variables $Re\, z, Im\, z$. The third-order polynomial contains a parameter $L_{\mathcal{H}}$ which is supposed to be an estimate of the Lipschitz constant of the Hessian. As this constant is not known, it has to be estimated by adapting it in a way that the produced sequence of iterates has monotonely decreasing function values. The step of the Cubic Newton is relatively expensive. Moreover, since the complex structure is not used explicitly, all four blocks of the Hessian are computed and enter in the formula for the iteration:

$$f(z_1) \leq \underbrace{f(z) + 2\left\langle g_{\bar{z}}, \Delta_z \right\rangle + \left\langle \left(\mathcal{H}_{\bar{z}z} + \frac{1}{2}\left(\mathcal{H}_{zz} + \mathcal{H}_{\bar{z}\bar{z}}\right)\right)\Delta_z, \Delta_z \right\rangle + \frac{L_{\mathcal{H}}}{6}\delta^3}_{L_{\mathcal{H}} \text{ is selected first, bound is computed after finding } \delta \text{ and computing } z_1};$$

$$\delta^2 = \left\| \left(\mathcal{H}_{\bar{z}z} + \frac{1}{2}\left(\mathcal{H}_{zz} + \mathcal{H}_{\bar{z}\bar{z}}\right) + \frac{\delta L_{\mathcal{H}}}{4} I_n \right)^{-1} g_{\bar{z}} \right\|^2,$$

$$\delta \geq \frac{4}{L_{\mathcal{H}}} \max\left\{ -\lambda_{\min}\left(\mathcal{H}_{\bar{z}z} + \frac{1}{2}\left(\mathcal{H}_{zz} + \mathcal{H}_{\bar{z}\bar{z}}\right)\right), 0 \right\}, \text{ here } L_{\mathcal{H}} \text{ is fixed};$$

$$z_1 := z - \left(\mathcal{H}_{\bar{z}z} + \frac{1}{2}\left(\mathcal{H}_{zz} + \mathcal{H}_{\bar{z}\bar{z}}\right) + \frac{\delta L_{\mathcal{H}}}{4} I_n \right)^{-1} g_{\bar{z}};$$

$$c_1 := \begin{pmatrix} z_1 \\ \bar{z}_1 \end{pmatrix},$$

The second tested method is the *Simplified Cubic Newton*. It uses only the mixed derivative block $\mathcal{H}_{\bar{z}z}$ of the Hessian and can hence be considered as the cubic regularization analog of MNM. It acts according to the formula

$$f(z_1) \leq f(z) + 2\left\langle g_{\bar{z}}, \Delta_z \right\rangle + \left\langle \mathcal{H}_{\bar{z}z}\Delta_z, \Delta_z \right\rangle + L\delta^2\left(1 + \frac{\delta}{6}\right);$$

$$\delta^2 = \left\| \left(\mathcal{H}_{\bar{z}z} + L\left(1 + \frac{\delta}{4}\right) I_n \right)^{-1} g_{\bar{z}} \right\|^2, \ \delta \geq 4 \max\left\{ \frac{-\lambda_{\min}\left(\mathcal{H}_{\bar{z}z}\right)}{L} - 1, \ 0 \right\}; \tag{9}$$

$$z_1 := z - \left(\mathcal{H}_{\bar{z}z} + L\left(1 + \frac{\delta}{4}\right) I_n \right)^{-1} g_{\bar{z}}.$$

Here also a Lipschitz constant has to be estimated. However, the last row above shows that the Simplified Cubic Newton method is nothing else than a Regularized Mixed Newton Method (RMNM) with a specific regularization term of Levenberg-Marquardt type (i.e., proportional to the identity matrix) which depends on the mentioned Lipschitz constant. It has been shown theoretically that this regularization (in fact, any positive definite regularization) does not harm the advantageous behaviour of the MNM in the neighbourhood of critical points. This theory has been reported in the report submitted in February.

Block Simplified Cubic Newton. The block Simplified Cubic Newton (SCN), defined by the formula

$$z_{k+1,i} := z_{k,i} - \left(\mathcal{H}_{\bar{z}_i z_i}(z_k)\right)^{-1} g_{\bar{z}_i},$$

uses the same rule for step size tuning as the single-block version above. The main difference is the use of a block diagonal matrix instead of $\mathcal{H}_{\bar{z}z}$. Namely, we use a block diagonal matrix with blocks $\mathcal{H}_{\bar{z}_i z_i}$, where $z_i \in \mathbb{R}^{n_i}$ is part of vector variable z. This approach reduces the cost of computing the Hessian, as derivatives only with respect to a subset of variables are needed. The details of the splitting into subsets of parameters will be given below.

Damped Mixed Newton Method. We compare the aforementioned methods to two "classical" methods. One of them is the ordinary MNM with a damping coefficient. We recall that the pure MNM does not necessarily produce a relaxing sequence, i.e., the function values do not need to decrease monotonously. By introducing a damping coefficient (which decreases the step length by retaining the direction of the step) we can enforce a monotone behaviour. We can write this method in the following form:

$$z_1 := z - \mu(\mathcal{H}_{\bar{z}z} + \lambda I_n)^{-1} g_{\bar{z}},$$

where μ is such that $f(z_1) \leq f(z)$ and λ is large enough constant to obtain decrease.

Levenberg-Marquardt Mixed Newton Method. The second "classical" method is the one which showed best performance in practice at Huawei. This is a special case of the RMNM, where the regularization is proportional to the identity matrix. The multiplier at this identity matrix is chosen adaptively to guarantee the relaxing sequence property. This method can be given by the following formula:

$$z_1 := z - \mu(\mathcal{H}_{\bar{z}z} + \lambda \| \operatorname{vec}(\mathcal{H}_{\bar{z}z}) \|_\infty I_n)^{-1} g_{\bar{z}},$$

where λ is an adaptive parameter such that $f(z_1) \leq f(z)$.

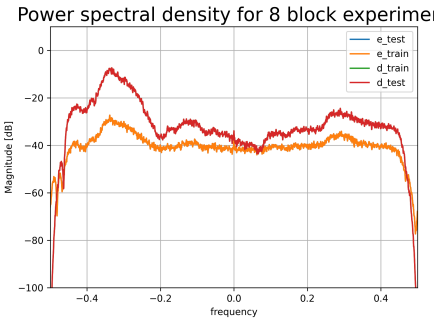
Numerical Results. In our experiments, we consider the model described on Fig. 9. We consider different numbers of branches (# br). For block methods, we consider a natural splitting where the parameters of each Nonlinear block (see Fig. 10) and FIR-P block (see Fig. 9) form a separate subset of variables. Note that the blocks of nonlinear layers contain an identical number of variables. As a consequence, this splitting is balanced in the sense that each minimization sub-step is over an equal number of parameters.

# br	Simplified CN	Cubic Newton	Block SCN	MNM (damped)	MNM (LM)
1	-0.0043	0.0000	-14.7051	-14.7034	-14.7034
4	-0.0042	-0.0000	-15.4061	-18.7032	-18.7033
8	-0.0157	-0.0000	-18.5142	-19.4819	-19.7225
16	-0.0194	-0.0000	-16.8293	-21.1601	-21.1600

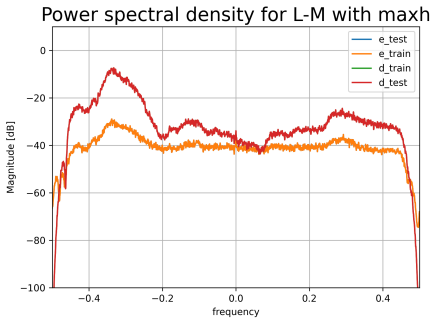
Table 5: NMSE (dB) after 1000 epochs

We run all methods for 1000 epochs and compare them with the Levenberg-Marquardt RMNM and the damped Mixed Newton Method. The results of this comparison are given in Table 5. We can see that the newly proposed Simplified CN and CN methods show a slow convergence or no convergence at all due to the non-convexity of the problem. At the same time, the use of splitting significantly improves quality. But this quality stills lags behind that of the Levenberg-Marquardt RMNM and the damped Mixed Newton Method, which show a comparable performance.

Besides, you can find spectrograms for the Block Simplified Cubic Newton and the Levenberg-Marquardt method on Figs. 11a and 11b. We can see that the results are similar.



(a) Block SCN



(b) MNM (LM)

Figure 11: Results of optimization methods for model with 8 branches

To summarize, of the newly introduced methods only the Block Simplified Cubic Newton can somehow compete with the favorites damped MNM and LM RMNM.

Additionally, we compare MNM and LM RMNM methods with the first order method - Gradient Descent. The result of this comparison for four branches can be found on Fig.12. We can see that methods outperform the first order method.

3.3 Delay Selection

The difficult part in training the two-layer model is the tuning of the discrete parameters, i.e., the delays in the different branches of the model. Many different approaches have been tested, but none of them led to a decisive breakthrough. Nevertheless, some of the approaches seem to be more promising than others.

3.3.1 Cross-entropy method for delays optimization

One of the approaches we suggested to optimize the discrete parameters of the model is based on a cross-entropy method which is an evolutionary algorithm for reinforcement learning. During the procedure two distributions of delays k_j and $l_{i,j}$ are simultaneously and independently adjusted based on the results of multiple experiments. The algorithm works as follows:

1. Two discrete distributions are initially set to be uniform on the possible range of delays.
2. Two sets of delays k_j, $j = 1, \dots, L$, and l_j, $j = 1, \dots, M$, are selected from their respective distributions without repetitions inside each set. Here l_j is a set of delays inside each non-linear block which is identical for all blocks: $\forall i \; l_{i,j} = l_j$.
3. The resulting model is trained (the lower-level optimization is performed) and evaluated.
4. Steps 2 and 3 are repeated multiple (T) times with different delays from the current distribution. Several (B) best models are selected, and the delay distributions are updated according to the frequency of occurrence of certain components in the best models.
5. The described procedure is repeated for the necessary number of steps until the distribution becomes close to a degenerate one (i.e., with smaller support).

In our experiments, we set the possible range of delays to be integers from -15 to 15. At each step, $T = 20$ models were constructed and trained and $B = 5$ of them were chosen to impact the distributions. Each continuous parameters training process was limited to 200 epochs and a threshold gradient norm of 10^{-3} for efficiency reasons. At each step, both distributions were updated according to the formula

$$P_{new} = (1 - \alpha)P_{old} + \alpha P_{step},$$

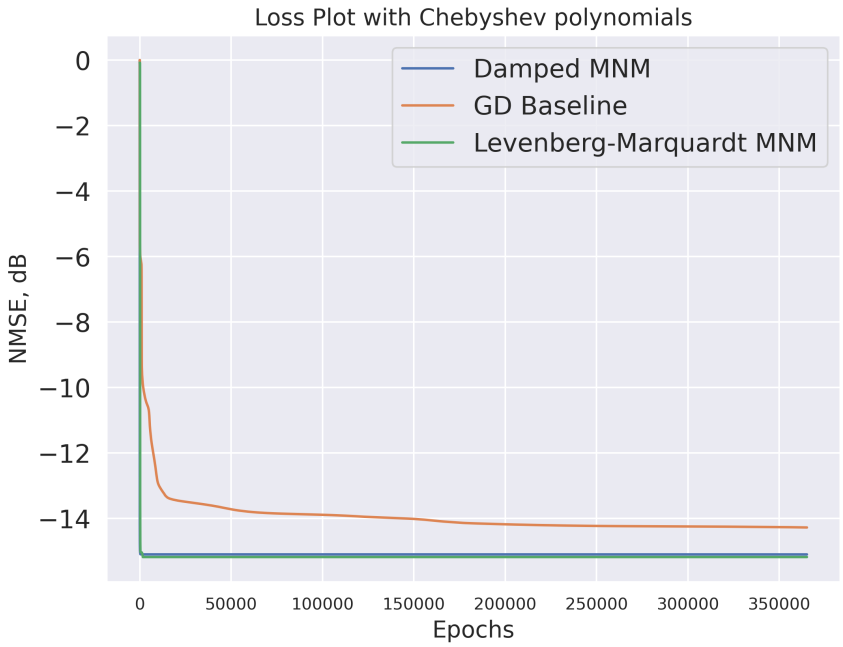


Figure 12: Comparison of Gradient Descent with MNM and LM RMNM

where P_{step} is the distribution of corresponding delays among the best models of the current step and $\alpha = 0.1$ is a parameter responsible for the distribution change rate. Experiments were conducted both on small models with $L = M = 3$ (in total 12 discrete parameters) and on bigger models with $L = 8$, $M = 3$ (32 parameters).

The results are presented in Figures 13–18. Figures 13–16 show how the distributions change over time. It can be seen that the distributions become more and more close to degenerate distributions, and the final distributions have common features. Thus, the largest probability density in all experiments falls on delays around 10. The distribution in Fig. 15 changed the least compared to the others because each model has 8 non-repeating delays from this distribution compared to 3 in all other cases. Figures 17 and 18 present the change in models' quality through the cross-entropy process steps. The average score of T models is improving while the best score among them does not change significantly.

To summarize, the cross-entropy rather discards bad models than finds new, better ones. Since all models in a sample have to be evaluated, it is relatively expensive.

3.3.2 Delay Selection Using Model Reduction

In each nonlinear block of the model, the signal is first passed through M delays, and then the results are mixed using coefficients from a matrix of size K x M. If the norms of the columns of this matrix are compared after training the continuous parameters, it can be judged which delay contributed the most to the final result. The idea of the method is to select delays based on their contribution to the large model. The algorithm is as follows:

1. A model is specified that has all possible delays from (31 delays -15 to 15) in each branch. The number of branches as well as the external delays for these branches are hyperparameters of the method.

2. The model is trained.

3. Based on the Euclidean norms of the matrix columns in each branch, some of the delays are discarded and a new model with fewer delays in each branch is created.

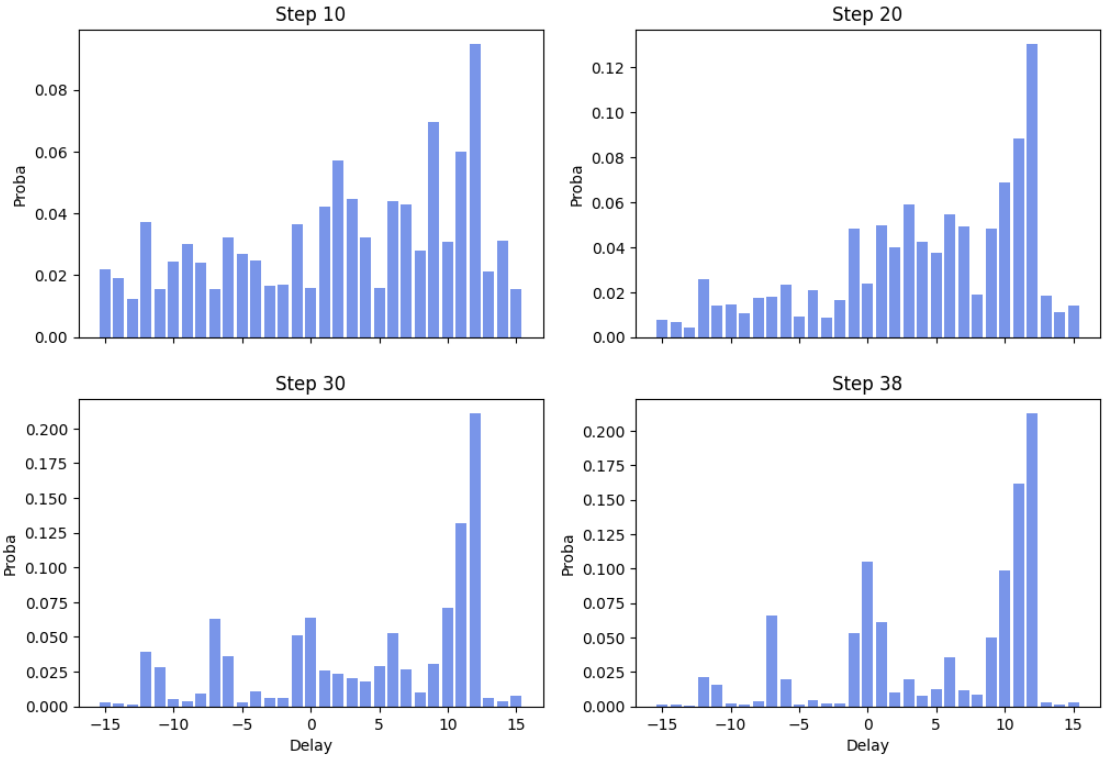


Figure 13: Change of k_j delays distribution during cross-entropy process for $L = 3$, $M = 3$

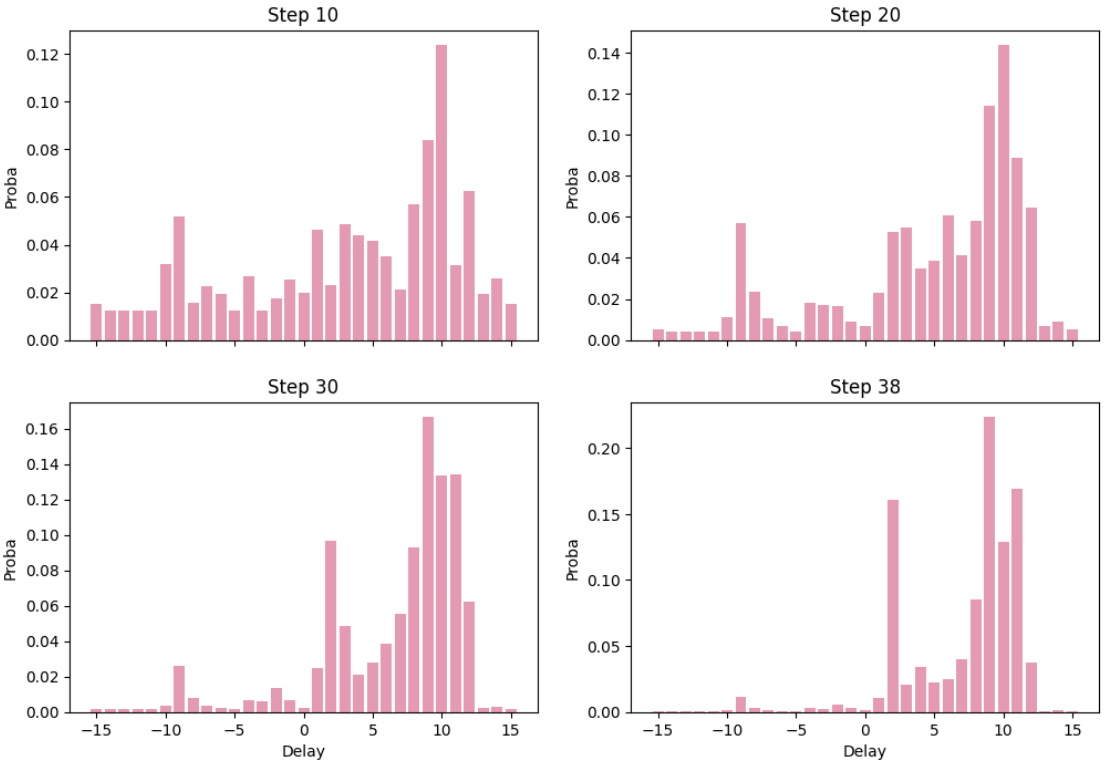


Figure 14: Change of l_j delays distribution during cross-entropy process for $L = 3$, $M = 3$

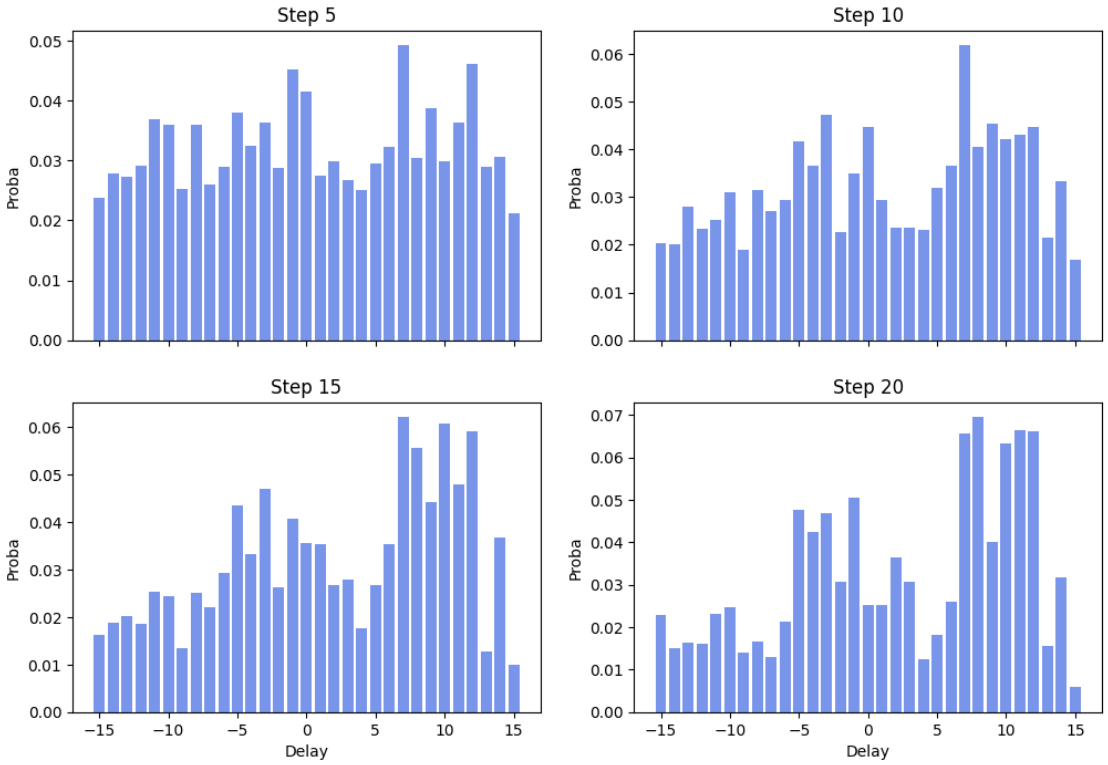


Figure 15: Change of k_j delays distribution during cross-entropy process for $L = 8$, $M = 3$

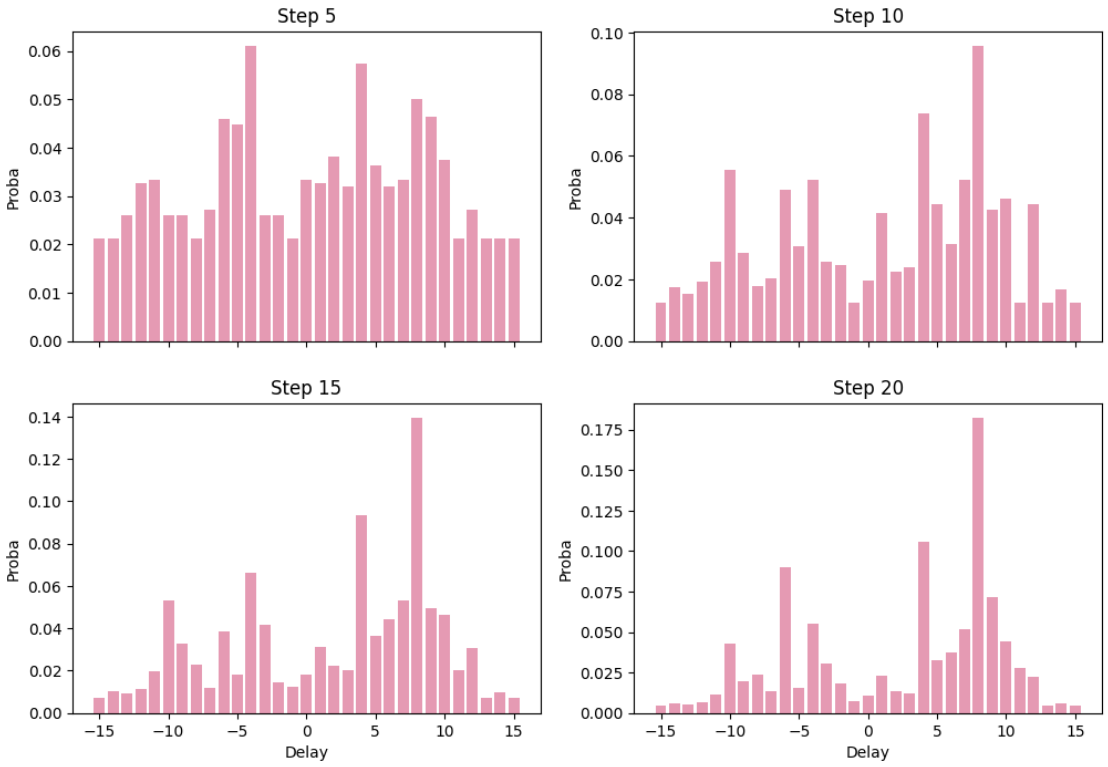


Figure 16: Change of l_j delays distribution during cross-entropy process for $L = 8$, $M = 3$

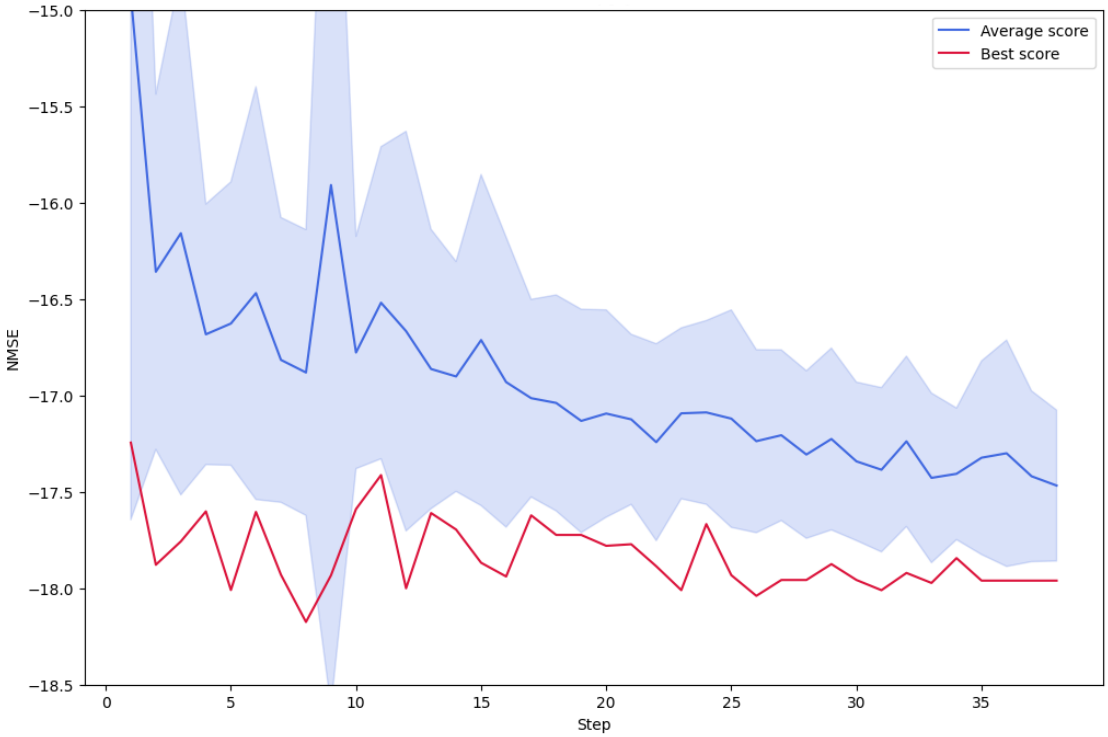


Figure 17: Cross-entropy process for $L = 3$, $M = 3$

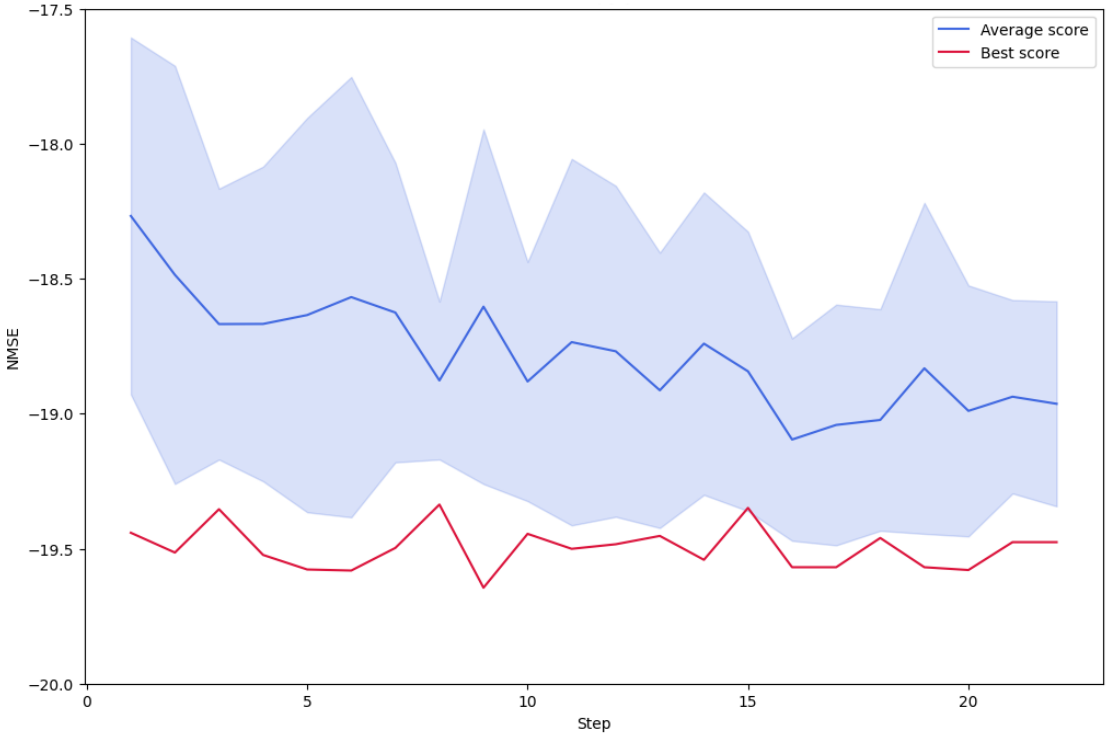


Figure 18: Cross-entropy process for $L = 8$, $M = 3$

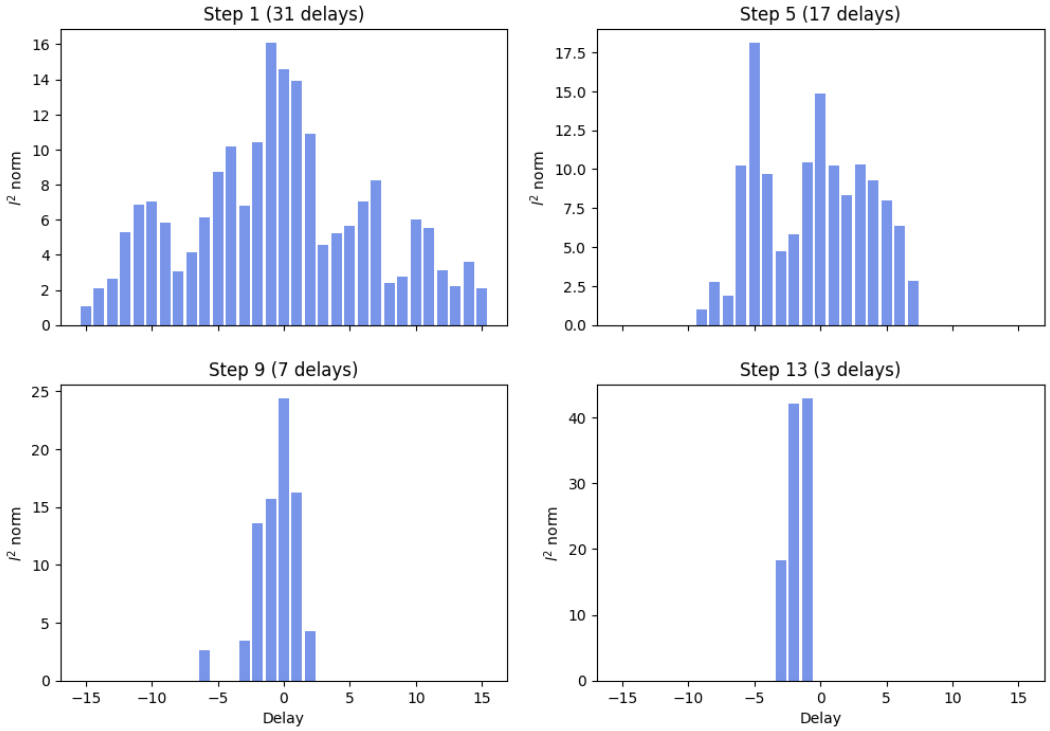


Figure 19: Distribution of column norms.One branch.

4. Steps 2 and 3 are repeated until the desired model size is achieved.

The selection of delays at each step can be done in several ways, for example:

1. Screen out a single delay that corresponds to the smallest norm.
2. Screen out a certain fraction of delays (e.g., 20%, but at least 1) that correspond to the smallest norm.
3. Screen out those delays whose norm does not exceed some threshold value, e.g., mean - std.

The first method unnecessarily lengthens the process, as training models, especially large ones at the initial stages of the algorithm, may require a lot of time and resources. The third method is not suitable for training models with multiple branches, as it may lead to the selection of different numbers of delays in different branches, which is not acceptable. Therefore, methods 2 and 3 were used in the single branch experiment, and only method 2 was used in the multi-branch experiment. The results are presented in Figures 19 through 21.

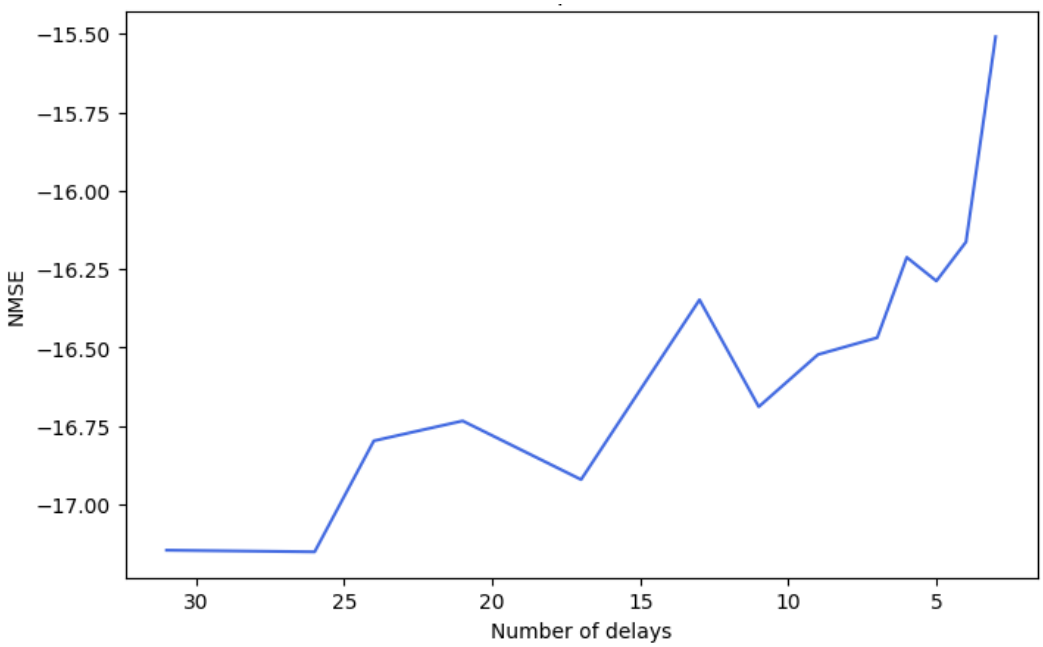


Figure 20: Model reduction process. One branch.

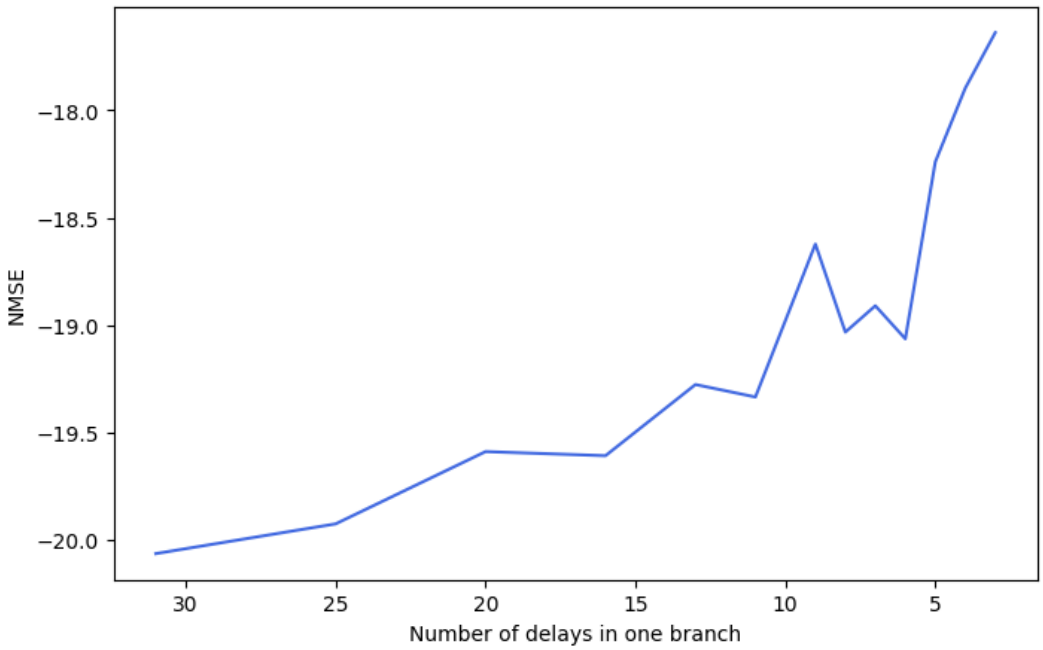


Figure 21: Model reduction process. Three branches.

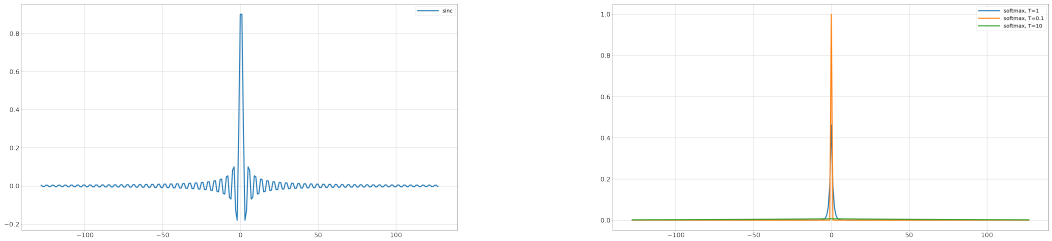
3.3.3 Continuous delay

This section is devoted to replacement of (discrete) delays blocks by continuous delays. Let us remind the details of such approach. If a vector x is a one-dimensional signal, then discrete delay can be given by the convolution $\mathcal{D}_k(x) = w * \S_i$ where $w_i = \delta_{i,k}$, k is a value of delay. The main idea of this approach is to replace a discrete delta function by some continuous parameterized function f with some parameters c, i.e. $w_i = f(i, c)$.

We want that this continuous function will be similar to discrete delay. In our experiments, we consider two strategy for our filters:

1. Sinc Filter: $w_i = \text{sinc}((i - c))$, where $c \in \mathbb{R}$ is the center of filter, and it approximates the value of delay
2. Softmax Filter: $w_i = \text{Softmax}\left(-\frac{|i-c|^p}{T}\right)$, where c is center of filter, T, p - internal parameters. Note, that $\lim\limits_{T\to 0} w_i = \delta_{i,k}$.

Note, in both case delays are differentiable with respect to center c. Examples of such filters can be found on Fig.22.



(a) Sinc Filter

(b) Softmax Filter

Figure 22: Filters

The Sinc filter can be trained by any optimization method. At the same time, Softmax filter needs a decrease of T parameter to approximate the initial discrete filter. To approach this we will start from $T = 100$ and further, we will decrease its parameter, during the optimization process. Namely, we will use strategy $T_k = \frac{100}{[k/100]+1}$, where k is a number of iteration of optimization method. You can find results of comparison of this strategy and taking $T = 10^{-1}$ for all iterations in Table 6.

#branches	p	const T	decrease T
1	$p = 1$	-8.51	-9.45
	$p = 2$	-8.54	-9.46
	$p = 3$	-8.51	-9.51
4	$p = 1$	-10.81	-14.85
	$p = 2$	-11.01	-14.92
	$p = 3$	-10.76	-14.88

Table 6: Comparison of NMSE (dB) for Sinc filter for different p and different strategies for T on models with 1 and 4 branches

We can see that train of delays with initially low T leads us to significantly lower quality. It can be explained by enough high Lipschitz constant of gradient for low T. At the same time, sequential decrease allows us to approach better quality. At the same time, value p does not affect quality significantly. Because of that, we will use $p = 2$ for future experiments.

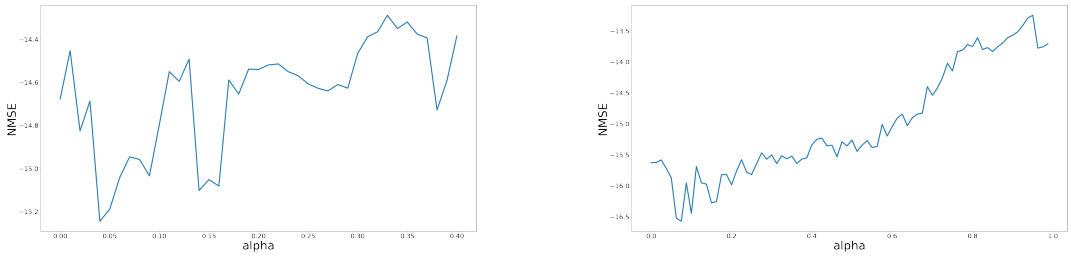
All filters were trained by MNM (damped). Besides, we tried to add noise for direction in this method. In other words, let us d_k and α_k are direction and step-size for kth iteration. Then our method has form $x_{k+1} = x_k + \alpha_k (d_k + n_k)$, where $n_k \sim \mathcal{N}(0, \frac{1}{(1+k)})$.

# br	Softmax Delay	Sinc Delay	Sinc+stoch	Baseline
1	-9.46	-10.75	-14.71	-14.70
4	-14.92	-14.73	-12.31	-18.70
8	-15.80	-15.68	-16.41	-19.72
16	-16.92	-17.21	-17.50	-21.16

Table 7: NMSE (dB) for different delays: 1) Softmax Delay - continuous delay with Softmax filter; 2) Sinc Delay - continuous delay with sinc filter; 3)Sinc+stoch - continuous delay with sinc filter trained with additive noise for direction; 4) Huawei Delay - baseline given by Huawei

We can see that all methods are significantly worse than the baseline from Huawei. On other hand, noise can avoid some local minima and improve quality. Nevertheless, results significantly depend on the starting point. Start from Huawei's baseline gives insignificant improvement.

Such negative results can be explained by the large number of local minima of the optimized function that was found in the previous report (see Fig. 23).



(a) 1 branch (b) 2 branch

Figure 23: Results for Sinc Filter for different fixed centers $c = c^* + \alpha d_c$, where c^* is the original discrete delay, d_c is random direction,

The results for Sinc Filter are presented in Fig. 23. You can find results for models with one and two branches. Firstly, we can see that in both cases, the surface of the loss function contains a big number of local minima. Besides, these points significantly differ in quality. Secondly, we see that for two branches we have a significant degradation of quality for an enough large distance from c^*.

3.3.4 Sequential Training of Delays

Let us note, that optimization problem can be given by the following form:

$$\min_{w\in\mathbb{R}^{n_w}, W_i\in\mathbb{R}^{n_i},, d_i\in\mathbb{Z}^4} \left\| w \star \mathcal{L}\left(\sum_{i=1}^{N} block(x, W_i, d_i)\right) - y \right\|^2,$$

where *block* is multiplication of signal with delay and output of nonlinear block, that uses some delays too, $\mathcal{L}$ is linear transform that does not contain trainable parameters.

In the previous section, one showed that continuous delay can improve quality for small models. But the full problem for models with large amounts of delays is too hard. Consequently, we propose the new approach. The main idea is to simplify the initial problem and train delays only for one block. Namely, in each step of our method, we will optimize delays only for one branch (see Algorithm 1).

In other words, in each iteration, we will train only one branch. It allows us significantly decreasing complexity of internal problem and search for good point for our continuous delays. We will use Sinc filter because it demonstrates better performance than Softmax Filter.

Note, that we cannot still solve internal problem exactly. So, each step of this algorithm leads us to the large number of iterations for internal optimization method. On the other hand, the internal problem contains a small number of parameters. Because of that, the training of model in such way

Data: y
Result: Delays $\{d_k\}_k$
$k \leftarrow 1$;
while $k \leq N$ **do**
  $w, W_k, d_k \leftarrow \arg \min_{w \in \mathbb{R}^{n_w}, W_k \in \mathbb{R}^{n_k}, , d_k \in \mathbb{Z}^4} \left\| w \star \mathcal{L} \left(\sum_{i=1}^{k} block(x, W_i, d_i) \right) - y \right\|^2,$
end

Algorithm 1: Sequential Training

does not require too much time. Namely, it does not slower by more than three times in comparison with usual training.

# br	SeqTain	Sinc Delay	Baseline
4	-15.72	-12.31	-18.70
8	-19.31	-16.41	-19.72
16	-17.30	-17.50	-21.16

Table 8: NMSE (dB) for different delays: 1) SeqTrain - sequential training for sinc filter; 2) Sinc Delay - Sinc filter trained with additive noise for direction; 3) Huawei Delay - baseline given by Huawei

The results of this method are presented in Table 8. Note, that the weights were fine-tuned after Sequential Training. We can find out that this approach improves quality for models with 4 and 8 branches in comparison with initial continuous delay training. On the other hand, it does not hold for model with 16 branches. This model is too hard for optimization in such a way. In some experiments, one can see improvement, but this improvement is not significant.

At the same time, this method is significantly worse than baseline delays. It especially holds for a large model with 16 branches. Nevertheless, it allows to improve quality of continuous delay.

3.3.5 Local Search for Delays

The previous approaches demonstrate that optimization problem with the continuous delays has too much local minima. In this section, we propose a local search to improve the baseline. The main idea is to choose some delay and to find the best delay in its neighborhood. After that, we move to the next delay. We continue this procedure until we cannot obtain some improvement.

Data: Target vector y, initial delays $\{d_k\}_{k=1}^N$, neighborhood construction $\mathcal{U} : \mathbb{Z} \to 2^{\mathbb{Z}}$
Result: Updated delays $\{d_k\}_k$
$k \leftarrow 1$;
while *True* **do**
  $d_{\text{old}} \leftarrow d$
  for $k \leq N, i \leq 4$ **do**
    $d_{k,i} \leftarrow \arg \min_{d_{k,i} \in \mathcal{U}(d_{k,i})} \left[\min_{w \in \mathbb{R}^{n_w}, W_i \in \mathbb{R}^{n_i}, i = \overline{1,N}} \left\| w \star \mathcal{L} \left(\sum_{i=1}^{N} block(x, W_i, d_i) \right) - y \right\|^2 \right],$
  end
  if $d_{old} = d$ **then**
    return d
  end
end

Algorithm 2: Local Search for Delay

In our experiments, we considered only the two nearest values for each delay. In other words, we tried $d_i - 1, d_i, d_i + 1$ on each iteration. Note, that on each step of our algorithm, we need to train our model $4mN$ times, where m is a number of neighbors. For example, one step of this method is slower 64 times than one train of model. It significantly restricts setting that can be tried.

To avoid this restriction, we decrease the number of epochs per step. It makes our method more inexact but it allows significantly decrease required time. After this method, we find tune continuous parameters of the obtained model.

Another way to decrease the required time is to ignore delays in nonlinear blocks. Our experiments demonstrate that separate delay in nonlinear block affect less than one delay not from this block. It is significant for large models. The results of Local Search can be found in Table 9.

# br	Local Search	LS (w/o nonlinear)	Baseline
1	-14.70	-14.70	-14.70
4	-18.84	-18.83	-18.70
8	-	-19.86	-19.72

Table 9: NMSE (dB) for different delays: 1) Local Search - local search for each delay; 2) Local Search (w/o nonlinear) - local search when delays in nonlinear block were ignored; 3) Huawei Delay - baseline given by Huawei

We can find out that the difference for local search with and without delays in nonlinear blocks does not differ significantly. Possibly, we see such phenomena because of too small considered neighborhood. At the same time, we consider that improves quality for model with 4 and 8 branches. Finally, note that experiments for 16 branches does not demonstrate improvement for considered epochs per iteration. Possibly, it was caused by too small a neighborhood but experiments for this model are too hard.

To sum up, Local Search can sometimes improve quality but the obtained improvement is non-significant. At the same time, it has enough high computational complexity.